

Start coding or [generate](#) with AI.

▼ Problem Statement

▼ Business Context

Renewable energy sources play an increasingly important role in the global energy mix, as the effort to reduce the environmental impact of energy production increases.

Out of all the renewable energy alternatives, wind energy is one of the most developed technologies worldwide. The U.S Department of Energy has put together a guide to achieving operational efficiency using predictive maintenance practices.

Predictive maintenance uses sensor information and analysis methods to measure and predict degradation and future component capability. The idea behind predictive maintenance is that failure patterns are predictable and if component failure can be predicted accurately and the component is replaced before it fails, the costs of operation and maintenance will be much lower.

The sensors fitted across different machines involved in the process of energy generation collect data related to various environmental factors (temperature, humidity, wind speed, etc.) and additional features related to various parts of the wind turbine (gearbox, tower, blades, break, etc.).

▼ Objective

"ReneWind" is a company working on improving the machinery/processes involved in the production of wind energy using machine learning and has collected data of generator failure of wind turbines using sensors. They have shared a ciphered version of the data, as the data collected through sensors is confidential (the type of data collected varies with companies). Data has 40 predictors, 20000 observations in the training set and 5000 in the test set.

The objective is to build various classification models, tune them, and find the best one that will help identify failures so that the generators could be repaired before failing/breaking to reduce the overall maintenance cost. The nature of predictions made by the classification model will translate as follows:

- True positives (TP) are failures correctly predicted by the model. These will result in repairing costs.
- False negatives (FN) are real failures where there is no detection by the model. These will result in replacement costs.
- False positives (FP) are detections where there is no failure. These will result in inspection costs.

It is given that the cost of repairing a generator is much less than the cost of replacing it, and the cost of inspection is less than the cost of repair.

"1" in the target variables should be considered as "failure" and "0" represents "No failure".

▼ Data Description

The data provided is a transformed version of the original data which was collected using sensors.

- Train.csv - To be used for training and tuning of models.
- Test.csv - To be used only for testing the performance of the final best model.

Both the datasets consist of 40 predictor variables and 1 target variable.

Please read the instructions carefully before starting the project.

This is a commented Jupyter IPython Notebook file in which all the instructions and tasks to be performed are mentioned.

- Blanks ***'are provided in the notebook that needs to be filled with an appropriate code to get the correct result. With every '*** blank, there is a comment that briefly describes what needs to be filled in the blank space.
- Identify the task to be performed correctly, and only then proceed to write the required code.
- Fill the code wherever asked by the commented lines like "# write your code here" or "# complete the code". Running incomplete code may throw error.

- Please run the codes in a sequential manner from the beginning to avoid any unnecessary errors.
- Add the results/observations (wherever mentioned) derived from the analysis in the presentation and submit the same.

✓ Installing and Importing the necessary libraries

```
# Installing the libraries with the specified version
!pip install --no-deps tensorflow==2.19.0 scikit-learn==1.6.1 matplotlib==3.10.0 seaborn==0.13.2 numpy==2.0.2 panda
```

Note:

- After running the above cell, kindly restart the runtime (for Google Colab) or notebook kernel (for Jupyter Notebook), and run all cells sequentially from the next cell.
- On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in **this notebook**.

```
# Library for data manipulation and analysis.
import pandas as pd
# Fundamental package for scientific computing.
import numpy as np
#splitting datasets into training and testing sets.
from sklearn.model_selection import train_test_split
#Imports tools for data preprocessing including label encoding, one-hot encoding, and standard scaling
from sklearn.preprocessing import LabelEncoder, OneHotEncoder,StandardScaler
#Imports a class for imputing missing values in datasets.
from sklearn.impute import SimpleImputer
#Imports the Matplotlib library for creating visualizations.
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
# Imports the Seaborn library for statistical data visualization.
import seaborn as sns
# Time related functions.
import time
#Imports functions for evaluating the performance of machine learning models
from sklearn.metrics import confusion_matrix, f1_score,accuracy_score, recall_score, precision_score, classification
#Imports metrics from
from sklearn import metrics

#Imports the tensorflow,keras and layers.
import tensorflow
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dense, Input, Dropout,BatchNormalization
from tensorflow.keras import backend

# to suppress unnecessary warnings
import warnings
warnings.filterwarnings("ignore")
```

✓ Loading the Data

```
# uncomment and run the following lines for Google Colab
# from google.colab import drive
# drive.mount('/content/drive')
```

```
df = pd.read_csv("Train.csv")    # Complete the code to import the training data
df_test = pd.read_csv("Test.csv")    # Complete the code to import the test data
```

✓ Data Overview

The initial steps to get an overview of any dataset is to:

- observe the first few rows of the dataset, to check whether the dataset has been loaded properly or not

- get information about the number of rows and columns in the dataset
- find out the data types of the columns to ensure that data is stored in the preferred format and the value of each property is as expected.
- check the statistical summary of the dataset to get an overview of the numerical columns of the data

✓ Checking the shape of the dataset

```
# Checking the number of rows and columns in the training data
df.shape # Complete the code to print the shape of the train data
```

```
(20000, 41)
```

```
# Checking the number of rows and columns in the test data
df_test.shape # Complete the code to print the shape of the test data
```

```
(5000, 41)
```

```
# let's create a copy of the training data
data = df.copy()
```

```
# let's create a copy of the testing data
data_test = df_test.copy()
```

✓ Displaying the first few rows of the dataset

```
# let's view the first 5 rows of the data
data.head() # Complete the code to view the first five rows of the train data
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33
0	-4.464606	-4.679129	3.101546	0.506130	-0.221083	-2.032511	-2.910870	0.050714	-1.522351	3.761892	...	3.059700	-1.690440
1	3.365912	3.653381	0.909671	-1.367528	0.332016	2.358938	0.732600	-4.332135	0.565695	-0.101080	...	-1.795474	3.032780
2	-3.831843	-5.824444	0.634031	-2.418815	-1.773827	1.016824	-2.098941	-3.173204	-2.081860	5.392621	...	-0.257101	0.803550
3	1.618098	1.888342	7.046143	-1.147285	0.083080	-1.529780	0.207309	-2.493629	0.344926	2.118578	...	-3.584425	-2.577474
4	-0.111440	3.872488	-3.758361	-2.982897	3.792714	0.544960	0.205433	4.848994	-1.854920	-6.220023	...	8.265896	6.629213

5 rows × 41 columns

```
#viewing first 5 rows of the test data
data_test.head() # Complete the code to view the first five rows of the test data
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33
0	-0.613489	-3.819640	2.202302	1.300420	-1.184929	-4.495964	-1.835817	4.722989	1.206140	-0.341909	...	2.291204	-5.411388
1	0.389608	-0.512341	0.527053	-2.576776	-1.016766	2.235112	-0.441301	-4.405744	-0.332869	1.966794	...	-2.474936	2.493582
2	-0.874861	-0.640632	4.084202	-1.590454	0.525855	-1.957592	-0.695367	1.347309	-1.732348	0.466500	...	-1.318888	-2.997464
3	0.238384	1.458607	4.014528	2.534478	1.196987	-3.117330	-0.924035	0.269493	1.322436	0.702345	...	3.517918	-3.074085
4	5.828225	2.768260	-1.234530	2.809264	-1.641648	-1.406698	0.568643	0.965043	1.918379	-2.774855	...	1.773841	-1.501573

5 rows × 41 columns

✓ Checking the data types of the columns in the dataset

```
# let's check the data types of the columns in the dataset
data.info() # Complete the code to view the data types of the columns in the train data
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 41 columns):
```

#	Column	Non-Null Count	Dtype
0	V1	19982 non-null	float64
1	V2	19982 non-null	float64
2	V3	20000 non-null	float64
3	V4	20000 non-null	float64
4	V5	20000 non-null	float64
5	V6	20000 non-null	float64
6	V7	20000 non-null	float64
7	V8	20000 non-null	float64
8	V9	20000 non-null	float64
9	V10	20000 non-null	float64
10	V11	20000 non-null	float64
11	V12	20000 non-null	float64
12	V13	20000 non-null	float64
13	V14	20000 non-null	float64
14	V15	20000 non-null	float64
15	V16	20000 non-null	float64
16	V17	20000 non-null	float64
17	V18	20000 non-null	float64
18	V19	20000 non-null	float64
19	V20	20000 non-null	float64
20	V21	20000 non-null	float64
21	V22	20000 non-null	float64
22	V23	20000 non-null	float64
23	V24	20000 non-null	float64
24	V25	20000 non-null	float64
25	V26	20000 non-null	float64
26	V27	20000 non-null	float64
27	V28	20000 non-null	float64
28	V29	20000 non-null	float64
29	V30	20000 non-null	float64
30	V31	20000 non-null	float64
31	V32	20000 non-null	float64
32	V33	20000 non-null	float64
33	V34	20000 non-null	float64
34	V35	20000 non-null	float64
35	V36	20000 non-null	float64
36	V37	20000 non-null	float64
37	V38	20000 non-null	float64
38	V39	20000 non-null	float64
39	V40	20000 non-null	float64
40	Target	20000 non-null	int64

dtypes: float64(40), int64(1)
memory usage: 6.3 MB

- Converting Target column to float

```
data['Target'] = data['Target'].astype(float)
```

Now checking for test data

```
data_test.info() # Complete the code to view the data types of the columns in the test data
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 41 columns):
#   Column  Non-Null Count  Dtype
---  -
0    V1      4995 non-null    float64
1    V2      4994 non-null    float64
2    V3      5000 non-null    float64
3    V4      5000 non-null    float64
4    V5      5000 non-null    float64
5    V6      5000 non-null    float64
6    V7      5000 non-null    float64
7    V8      5000 non-null    float64
8    V9      5000 non-null    float64
9    V10     5000 non-null    float64
10   V11     5000 non-null    float64
11   V12     5000 non-null    float64
12   V13     5000 non-null    float64
13   V14     5000 non-null    float64
14   V15     5000 non-null    float64
15   V16     5000 non-null    float64
16   V17     5000 non-null    float64
17   V18     5000 non-null    float64
18   V19     5000 non-null    float64
19   V20     5000 non-null    float64
20   V21     5000 non-null    float64
```

```
21 V22      5000 non-null    float64
22 V23      5000 non-null    float64
23 V24      5000 non-null    float64
24 V25      5000 non-null    float64
25 V26      5000 non-null    float64
26 V27      5000 non-null    float64
27 V28      5000 non-null    float64
28 V29      5000 non-null    float64
29 V30      5000 non-null    float64
30 V31      5000 non-null    float64
31 V32      5000 non-null    float64
32 V33      5000 non-null    float64
33 V34      5000 non-null    float64
34 V35      5000 non-null    float64
35 V36      5000 non-null    float64
36 V37      5000 non-null    float64
37 V38      5000 non-null    float64
38 V39      5000 non-null    float64
39 V40      5000 non-null    float64
40 Target   5000 non-null    int64
dtypes: float64(40), int64(1)
memory usage: 1.6 MB
```

Converting Target to float

```
data_test['Target'] = data_test['Target'].astype(float)
```

✓ Checking for duplicate values

```
# let's check for duplicate values in the data
data.duplicated().sum() # Complete the code to check for duplicate values in the train data

np.int64(0)
```

✓ Checking for missing values

```
# let's check for missing values in the data
data.isnull().sum()
```

	0
V1	18
V2	18
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0
V11	0
V12	0
V13	0
V14	0
V15	0
V16	0
V17	0
V18	0
V19	0
V20	0
V21	0
V22	0
V23	0
V24	0
V25	0
V26	0
V27	0
V28	0
V29	0
V30	0
V31	0

```
# let's check for missing values in the test data
data_test.isnull().sum() #
```

V34	0
V35	0
V36	0
V37	0
V38	0
V39	0
V40	0
Target	0

dtype: int64

	0
V1	5
V2	6
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0
V11	0
V12	0
V13	0
V14	0
V15	0
V16	0
V17	0
V18	0
V19	0
V20	0
V21	0
V22	0
V23	0
V24	0
V25	0
V26	0
V27	0
V28	0
V29	0
V30	0
V31	0

✓ Statistical summary of the dataset

```
# let's view the statistical summary of the numerical columns in the data
data.describe().T # Complete the code to view the statistical summary of the train data
```

V36	0
V37	0
V38	0
V39	0
V40	0
Target	0

dtype: int64

	count	mean	std	min	25%	50%	75%	max
V1	19982.0	-0.271996	3.441625	-11.876451	-2.737146	-0.747917	1.840112	15.493002
V2	19982.0	0.440430	3.150784	-12.319951	-1.640674	0.471536	2.543967	13.089269
V3	20000.0	2.484699	3.388963	-10.708139	0.206860	2.255786	4.566165	17.090919
V4	20000.0	-0.083152	3.431595	-15.082052	-2.347660	-0.135241	2.130615	13.236381
V5	20000.0	-0.053752	2.104801	-8.603361	-1.535607	-0.101952	1.340480	8.133797
V6	20000.0	-0.995443	2.040970	-10.227147	-2.347238	-1.000515	0.380330	6.975847
V7	20000.0	-0.879325	1.761626	-7.949681	-2.030926	-0.917179	0.223695	8.006091
V8	20000.0	-0.548195	3.295756	-15.657561	-2.642665	-0.389085	1.722965	11.679495
V9	20000.0	-0.016808	2.160568	-8.596313	-1.494973	-0.067597	1.409203	8.137580
V10	20000.0	-0.012998	2.193201	-9.853957	-1.411212	0.100973	1.477045	8.108472
V11	20000.0	-1.895393	3.124322	-14.832058	-3.922404	-1.921237	0.118906	11.826433
V12	20000.0	1.604825	2.930454	-12.948007	-0.396514	1.507841	3.571454	15.080698
V13	20000.0	1.580486	2.874658	-13.228247	-0.223545	1.637185	3.459886	15.419616
V14	20000.0	-0.950632	1.789651	-7.738593	-2.170741	-0.957163	0.270677	5.670664
V15	20000.0	-2.414993	3.354974	-16.416606	-4.415322	-2.382617	-0.359052	12.246455
V16	20000.0	-2.925225	4.221717	-20.374158	-5.634240	-2.682705	-0.095046	13.583212
V17	20000.0	-0.134261	3.345462	-14.091184	-2.215611	-0.014580	2.068751	16.756432
V18	20000.0	1.189347	2.592276	-11.643994	-0.403917	0.883398	2.571770	13.179863
V19	20000.0	1.181808	3.396925	-13.491784	-1.050168	1.279061	3.493299	13.237742
V20	20000.0	0.023608	3.669477	-13.922659	-2.432953	0.033415	2.512372	16.052339
V21	20000.0	-3.611252	3.567690	-17.956231	-5.930360	-3.532888	-1.265884	13.840473
V22	20000.0	0.951835	1.651547	-10.122095	-0.118127	0.974687	2.025594	7.409856
V23	20000.0	-0.366116	4.031860	-14.866128	-3.098756	-0.262093	2.451750	14.458734
V24	20000.0	1.134389	3.912069	-16.387147	-1.468062	0.969048	3.545975	17.163291
V25	20000.0	-0.002186	2.016740	-8.228266	-1.365178	0.025050	1.397112	8.223389
V26	20000.0	1.873785	3.435137	-11.834271	-0.337863	1.950531	4.130037	16.836410
V27	20000.0	-0.612413	4.368847	-14.904939	-3.652323	-0.884894	2.189177	17.560404
V28	20000.0	-0.883218	1.917713	-9.269489	-2.171218	-0.891073	0.375884	6.527643
V29	20000.0	-0.985625	2.684365	-12.579469	-2.787443	-1.176181	0.629773	10.722055
V30	20000.0	-0.015534	3.005258	-14.796047	-1.867114	0.184346	2.036229	12.505812
V31	20000.0	0.486842	3.461384	-13.722760	-1.817772	0.490304	2.730688	17.255090
V32	20000.0	0.303799	5.500400	-19.876502	-3.420469	0.052073	3.761722	23.633187
V33	20000.0	0.049825	3.575285	-16.898353	-2.242857	-0.066249	2.255134	16.692486
V34	20000.0	-0.462702	3.183841	-17.985094	-2.136984	-0.255008	1.436935	14.358213
V35	20000.0	2.229620	2.937102	-15.349803	0.336191	2.098633	4.064358	15.291065
V36	20000.0	1.514809	3.800860	-14.833178	-0.943809	1.566526	3.983939	19.329576
V37	20000.0	0.011316	1.788165	-5.478350	-1.255819	-0.128435	1.175533	7.467006
V38	20000.0	-0.344025	3.948147	-17.375002	-2.987638	-0.316849	2.279399	15.289923
V39	20000.0	0.890653	1.753054	-6.438880	-0.272250	0.919261	2.057540	7.759877
V40	20000.0	-0.875630	3.012155	-11.023935	-2.940193	-0.920806	1.119897	10.654265
Target	20000.0	0.055500	0.228959	0.000000	0.000000	0.000000	0.000000	1.000000

▼ Exploratory Data Analysis



Univariate analysis

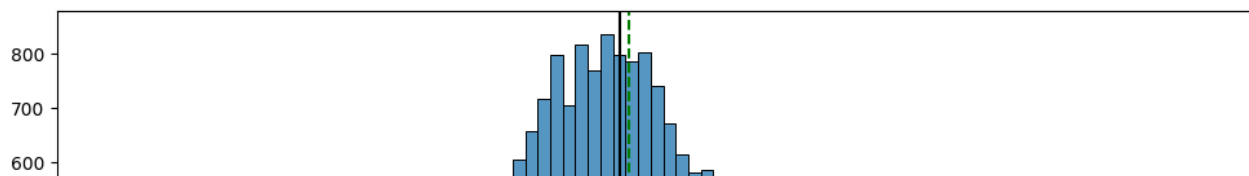
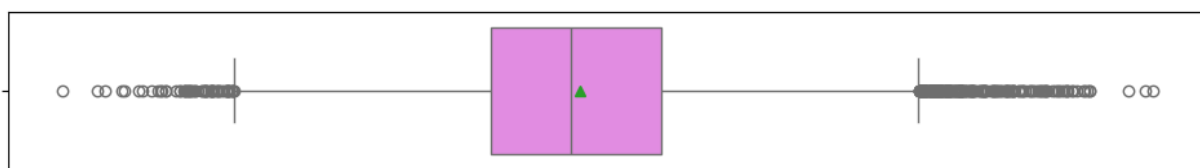
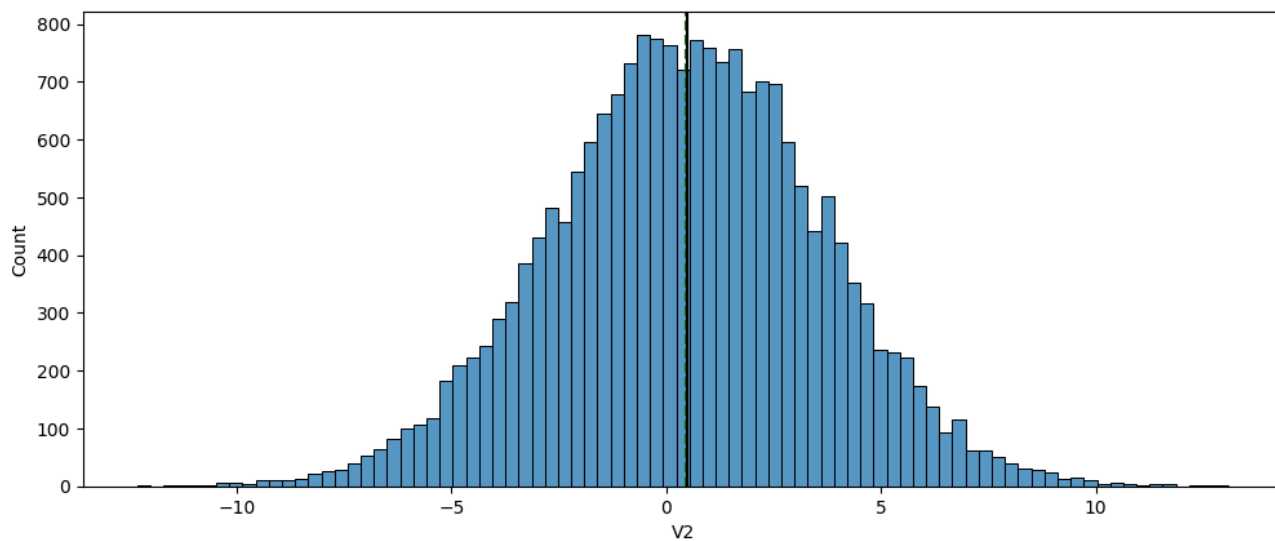
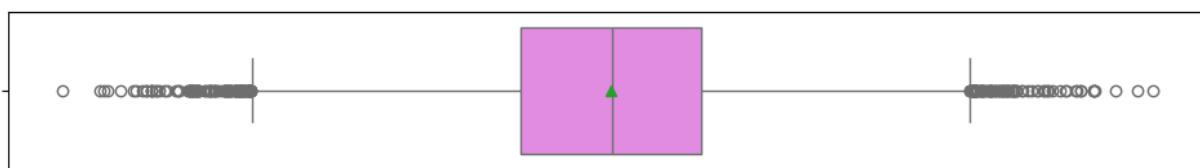
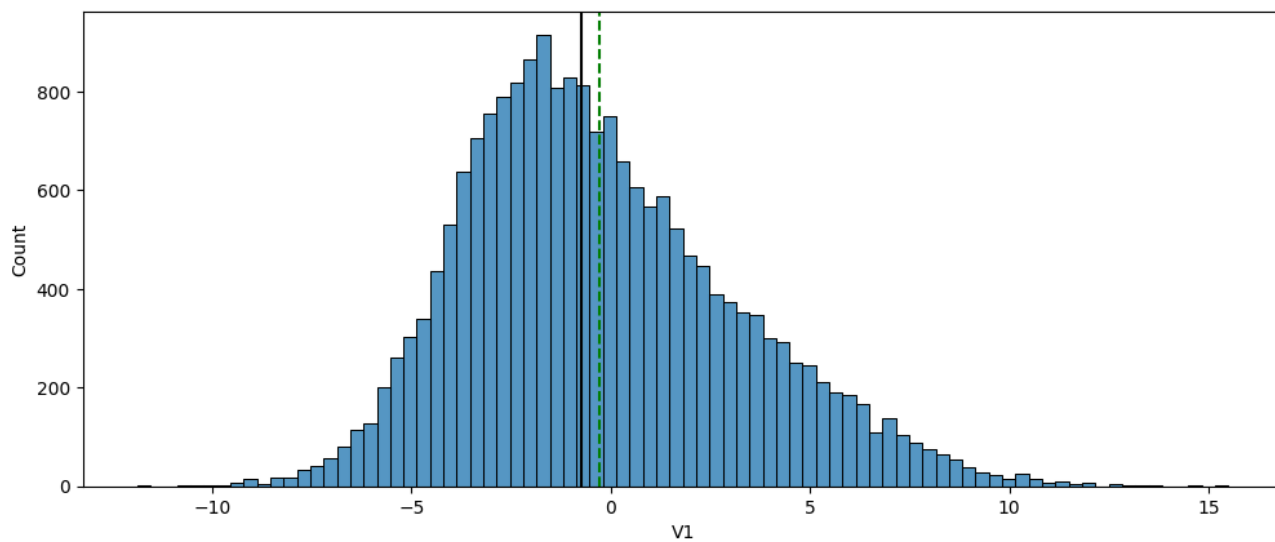
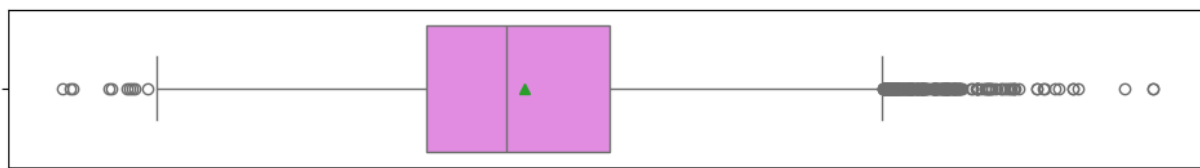
```
# function to plot a boxplot and a histogram along the same scale.

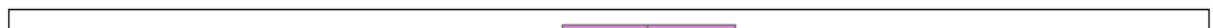
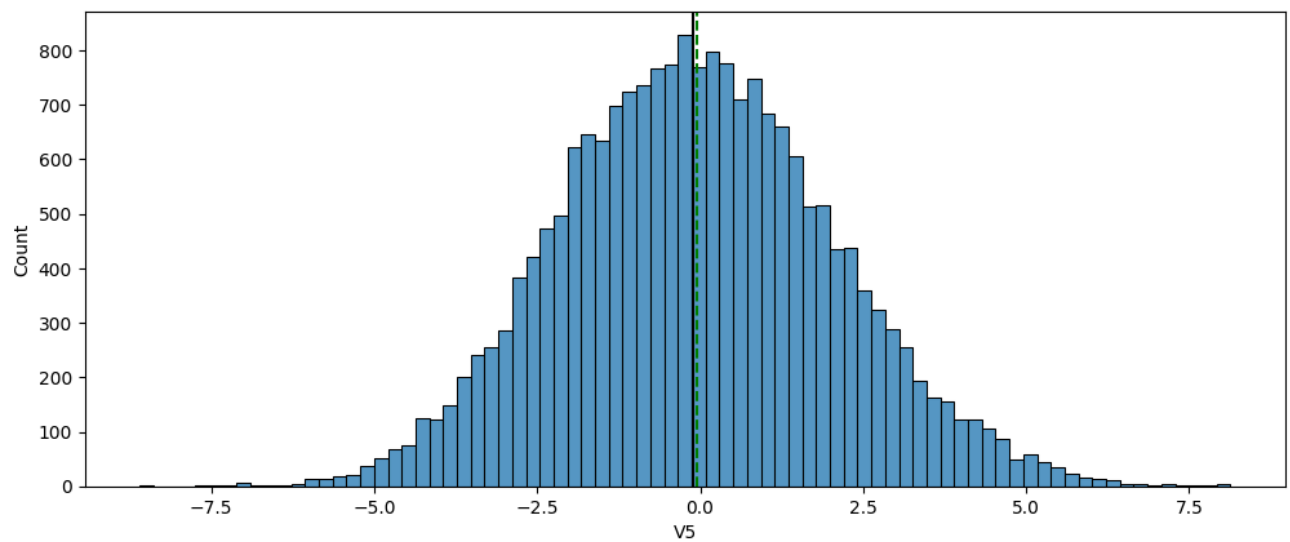
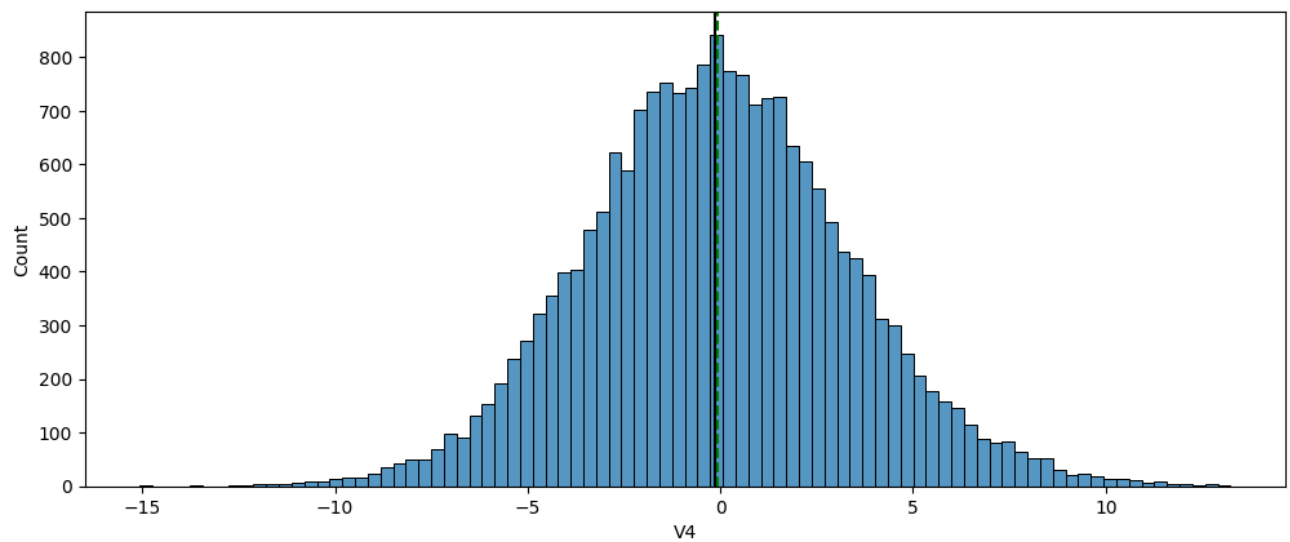
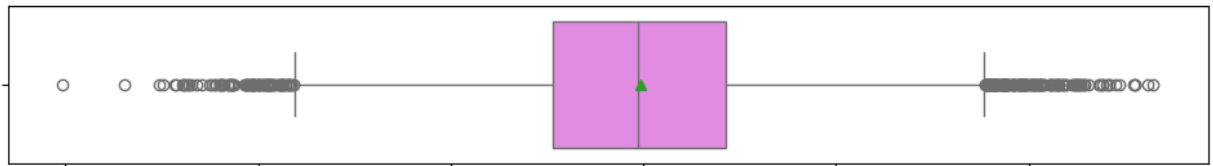
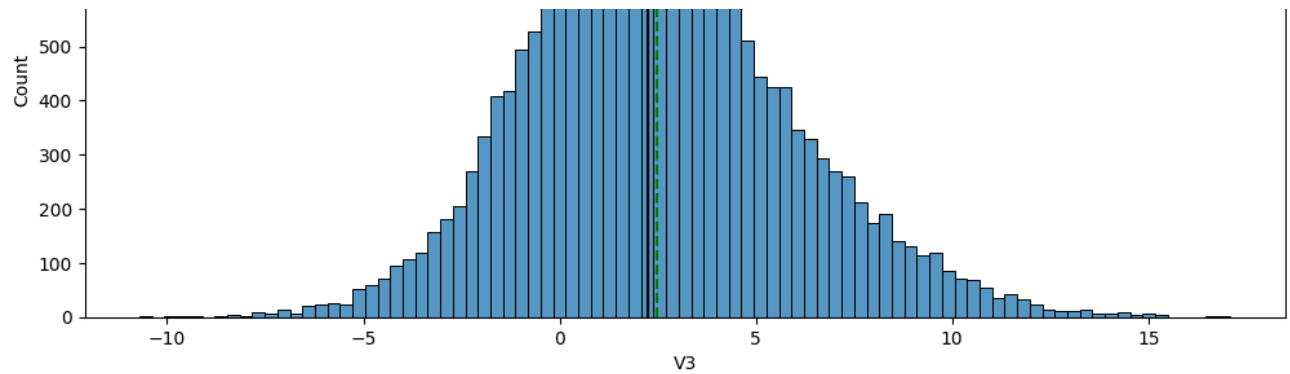
def histogram_boxplot(data, feature, figsize=(12, 7), kde=False, bins=None):
    """
    Boxplot and histogram combined

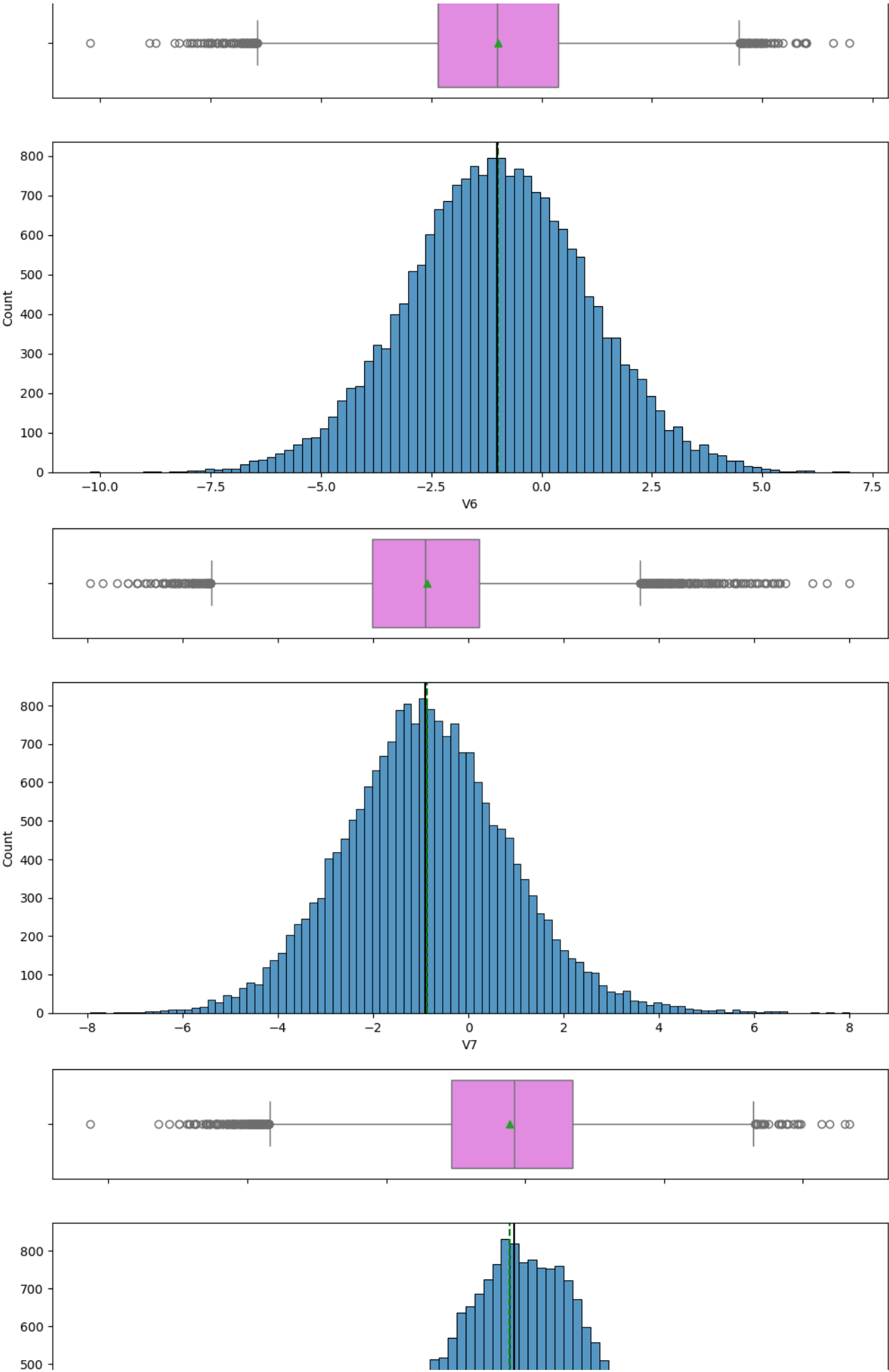
    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (12,7))
    kde: whether to the show density curve (default False)
    bins: number of bins for histogram (default None)
    """
    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2, # Number of rows of the subplot grid= 2
        sharex=True, # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    ) # creating the 2 subplots
    sns.boxplot(
        data= data, x= feature, ax=ax_box2, showmeans=True, color="violet"
    ) # boxplot will be created and a star will indicate the mean value of the column
    sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins, palette="winter"
    ) if bins else sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2
    ) # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color="green", linestyle="--"
    ) # Add mean to the histogram
    ax_hist2.axvline(
        data[feature].median(), color="black", linestyle="-"
    ) # Add median to the histogram
```

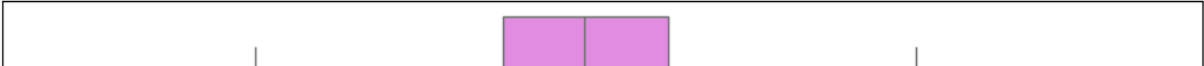
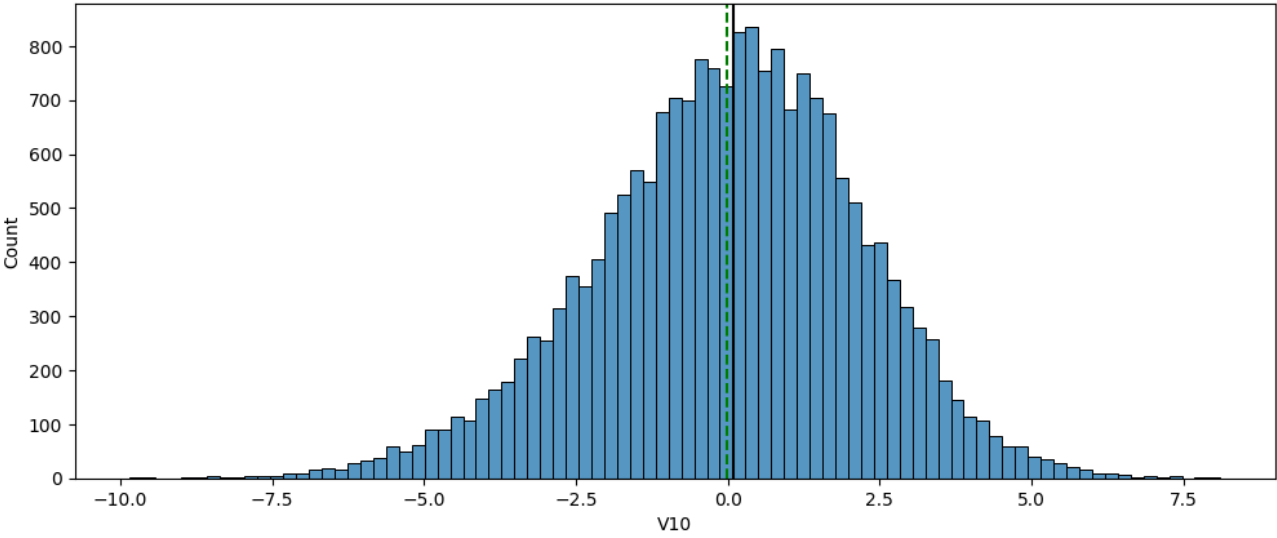
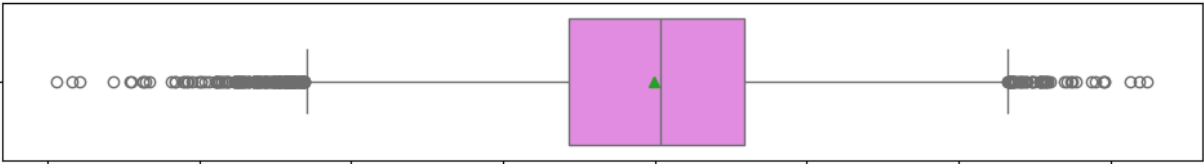
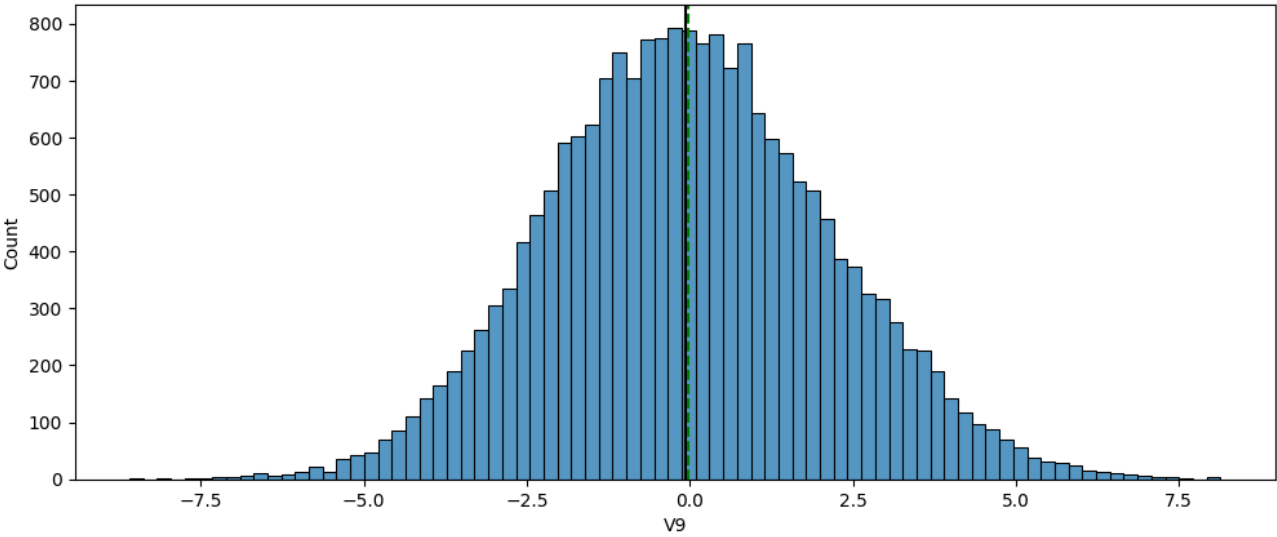
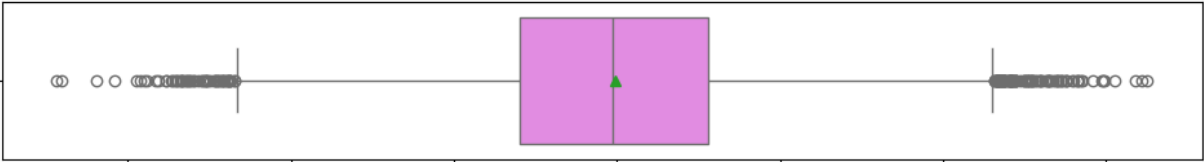
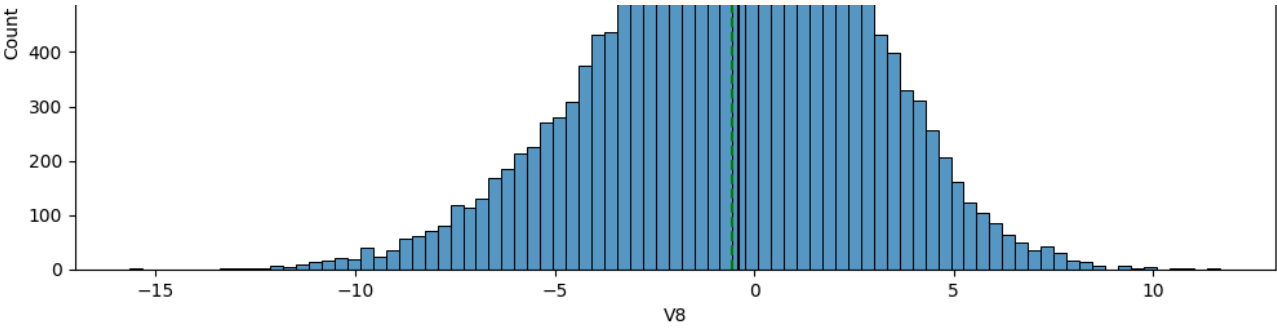
Variables V1 to V29

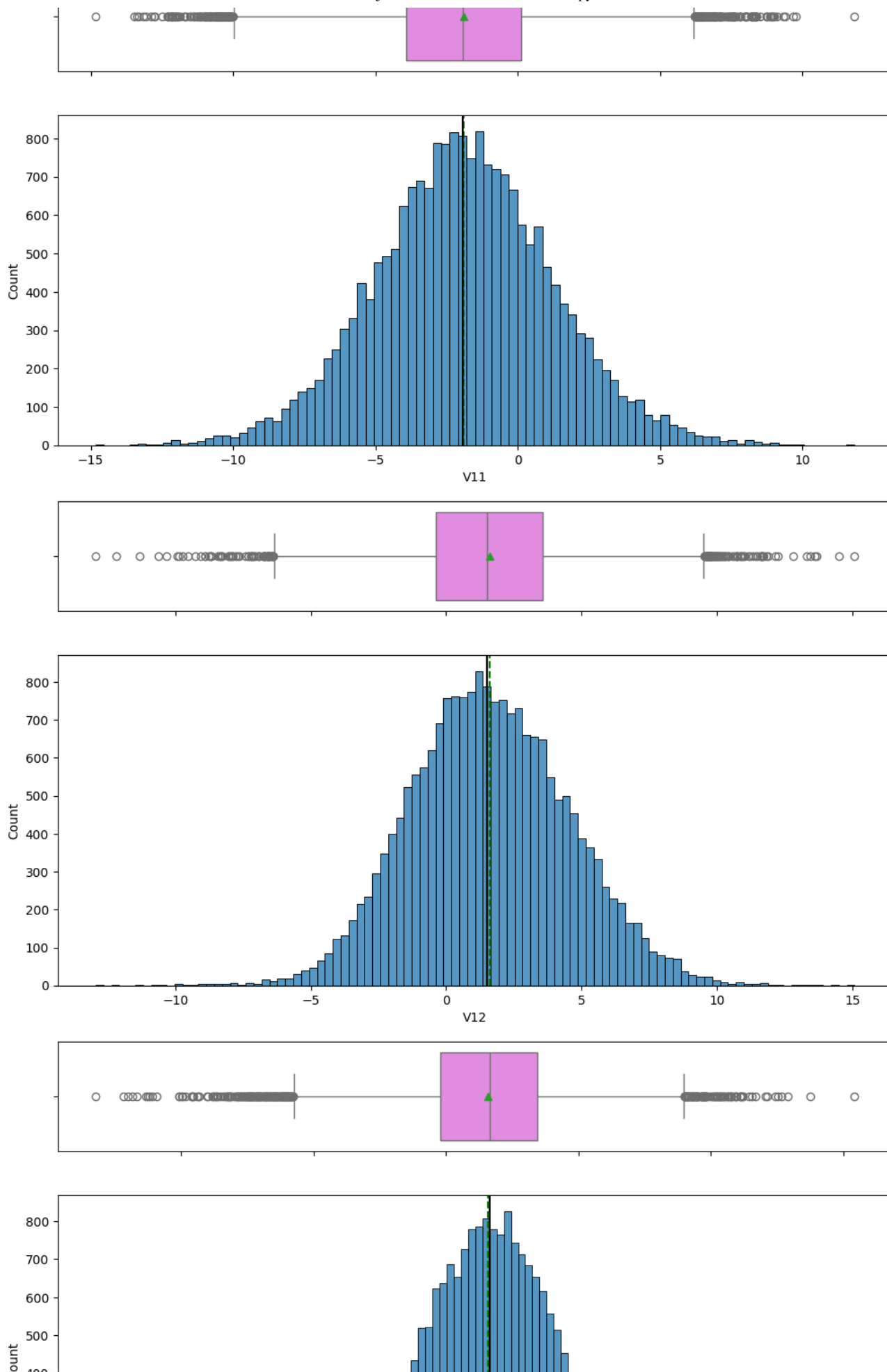
```
for feature in df.columns:
    histogram_boxplot(df, feature, figsize=(12, 7), kde=False, bins=None)
```

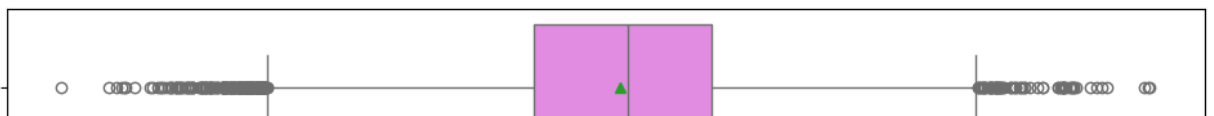
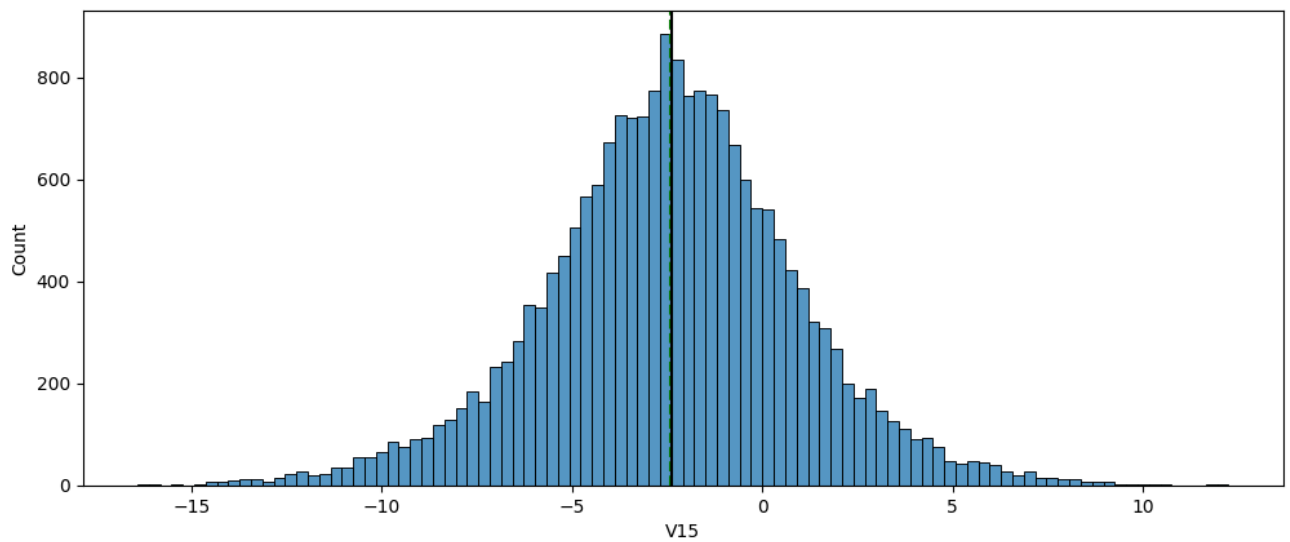
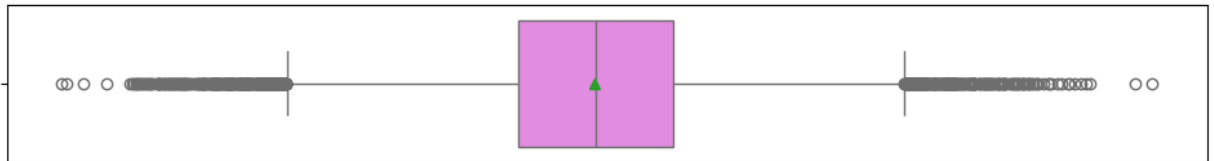
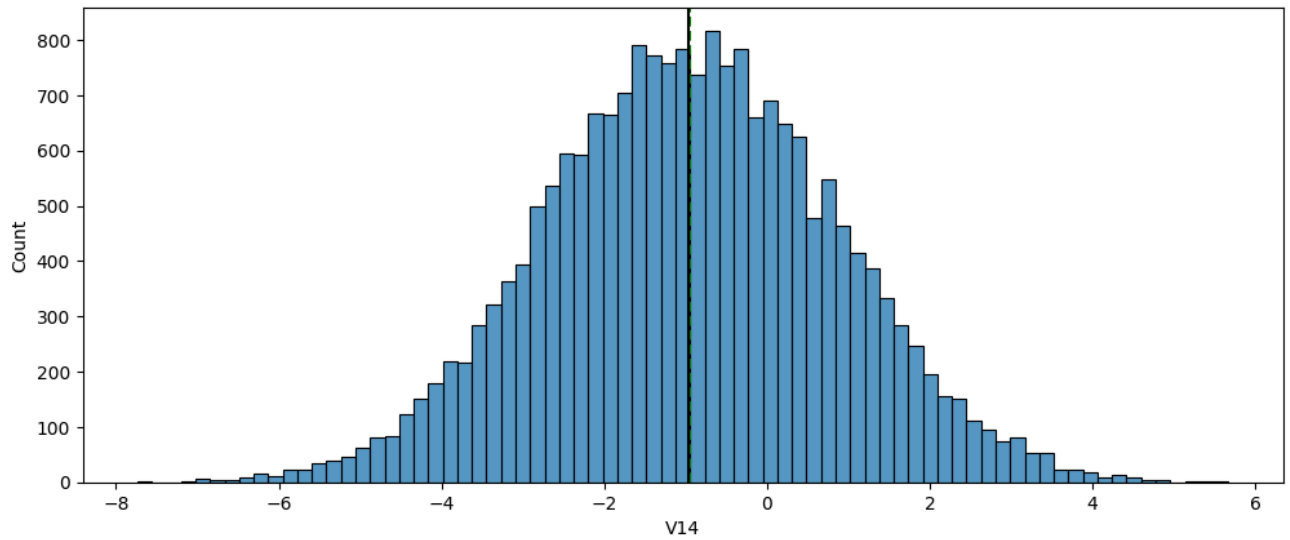
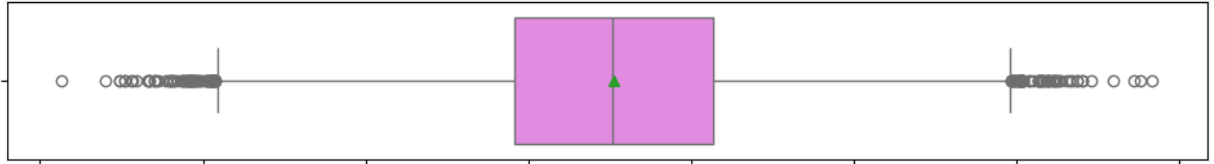
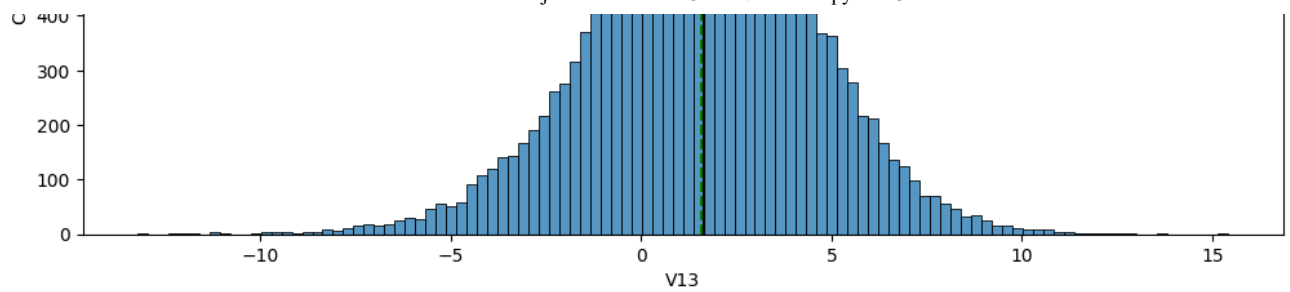



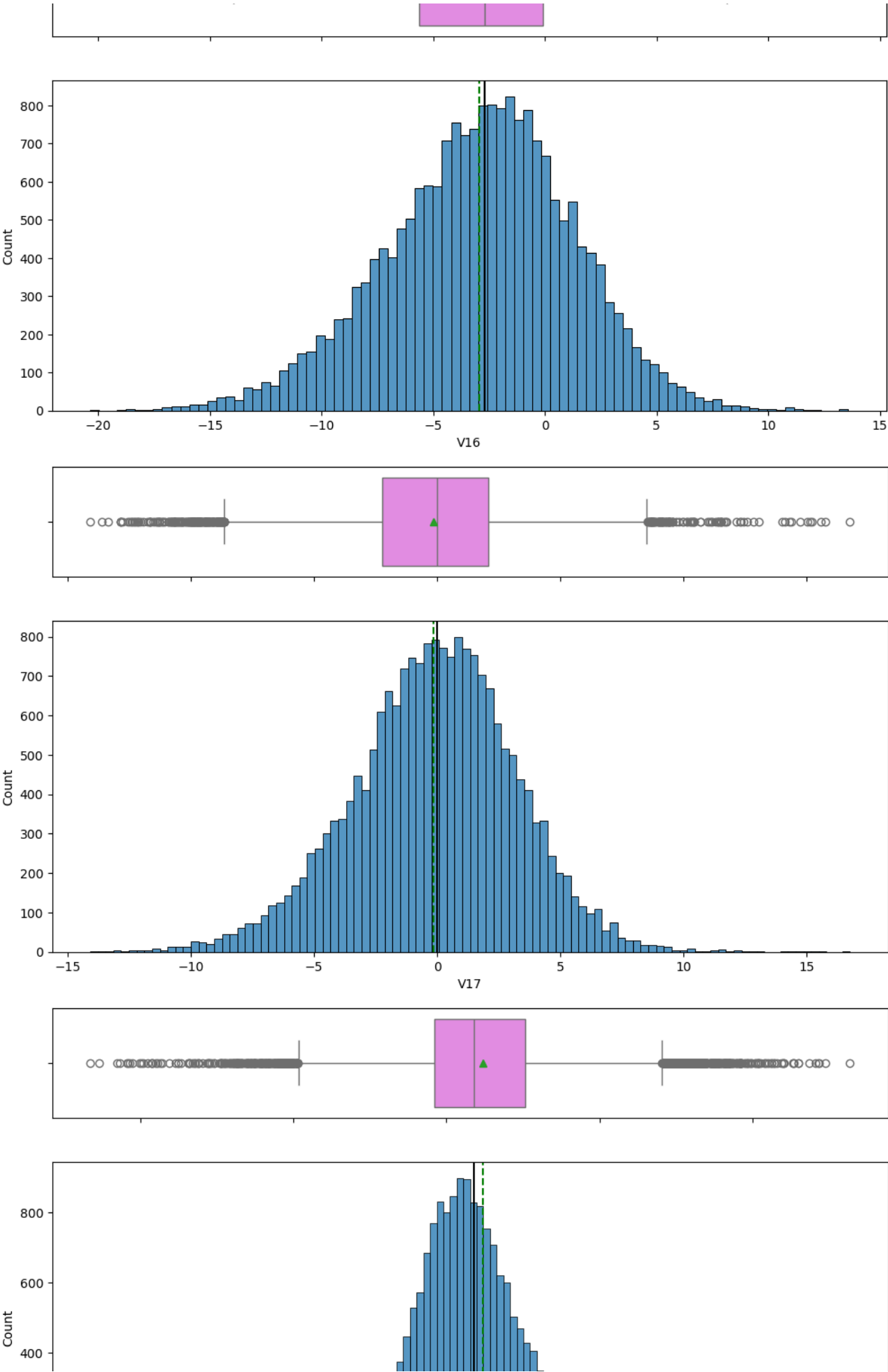


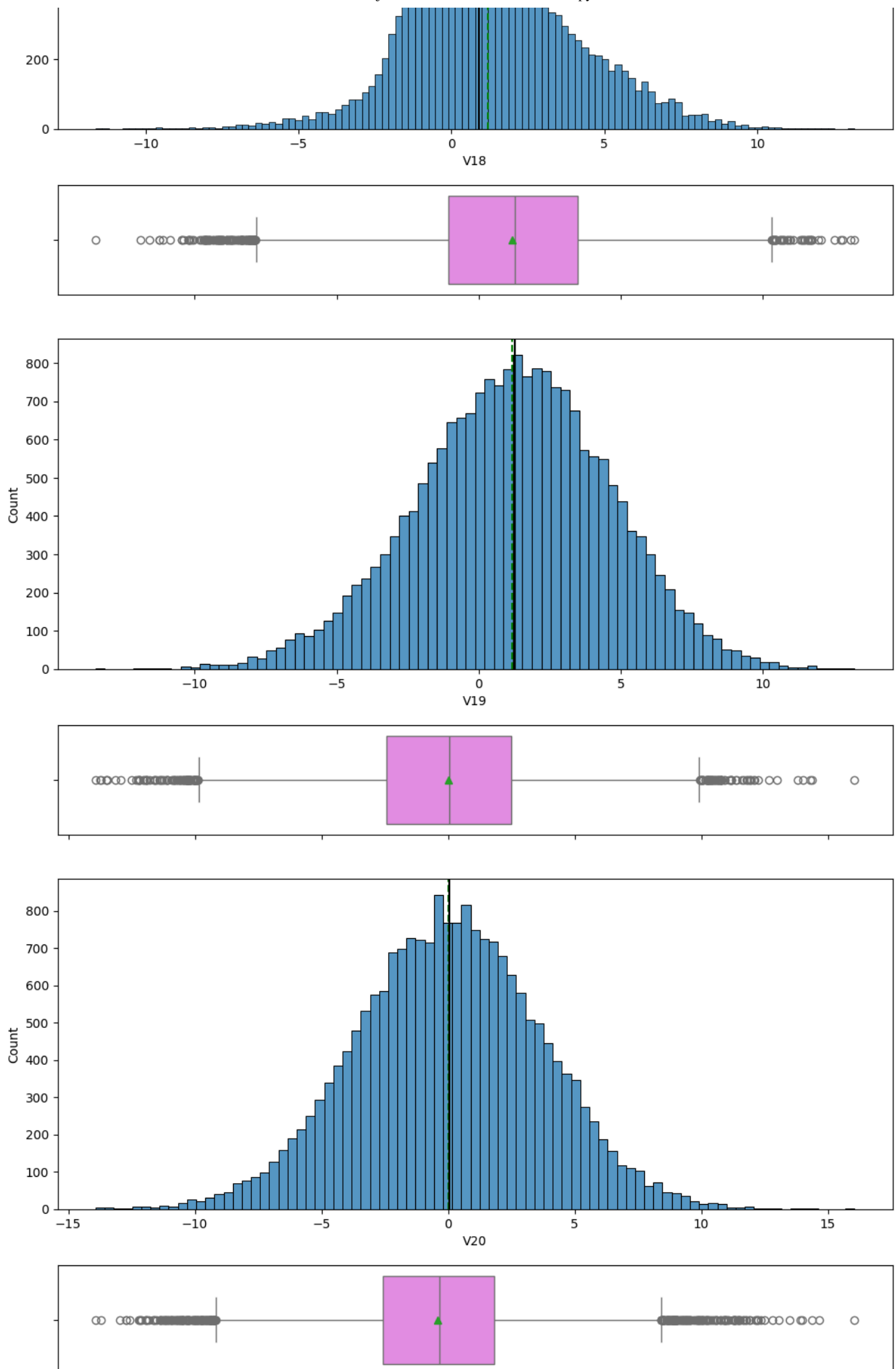


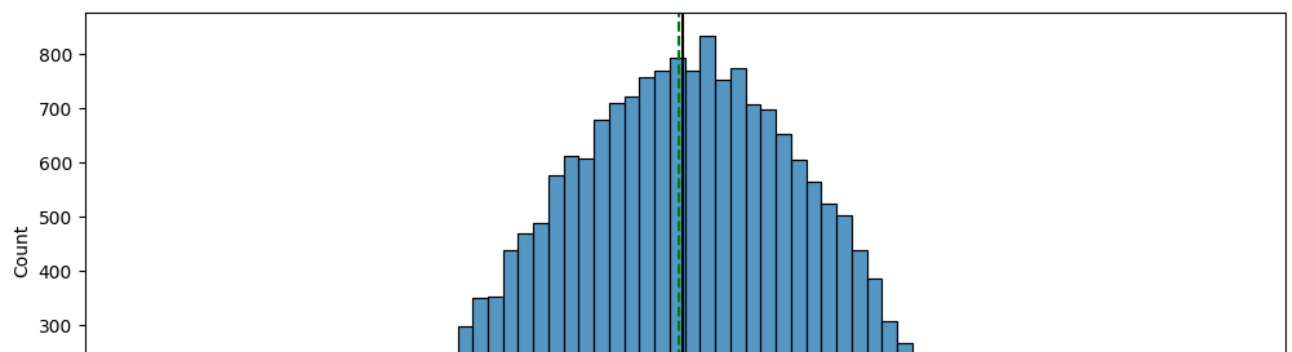
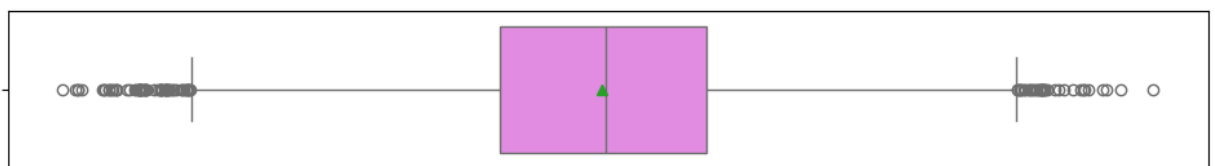
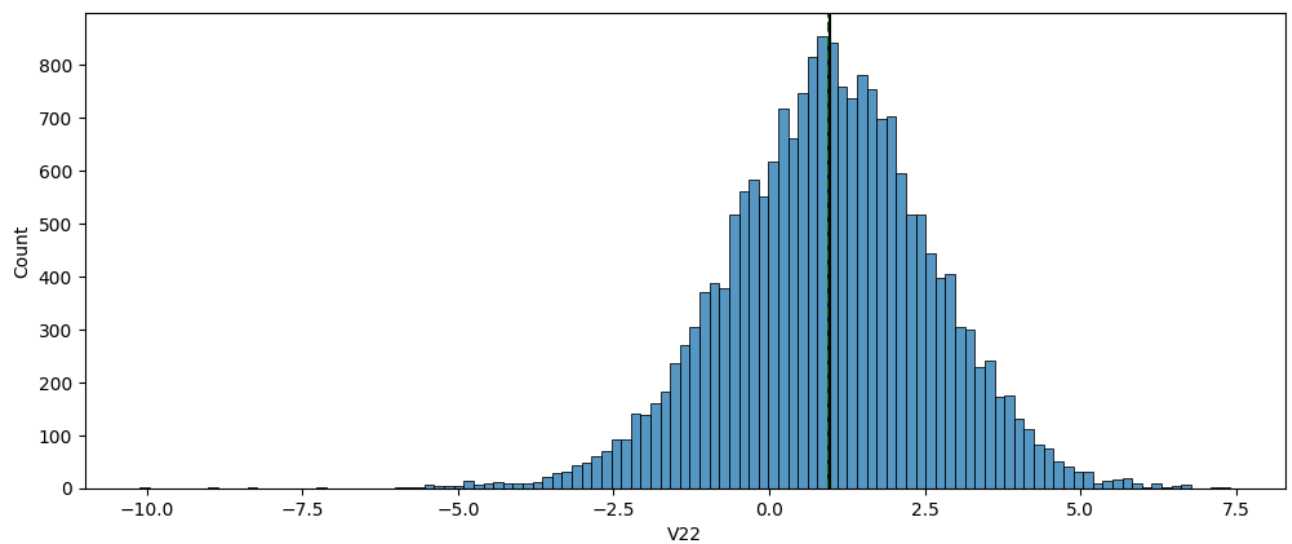
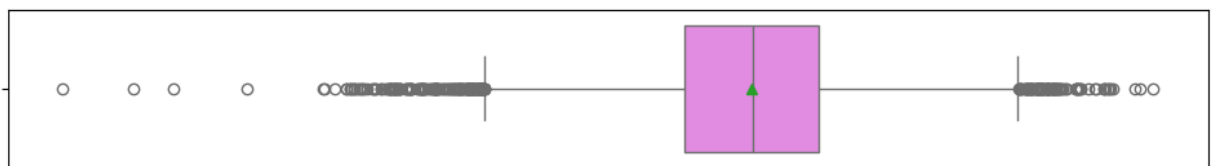
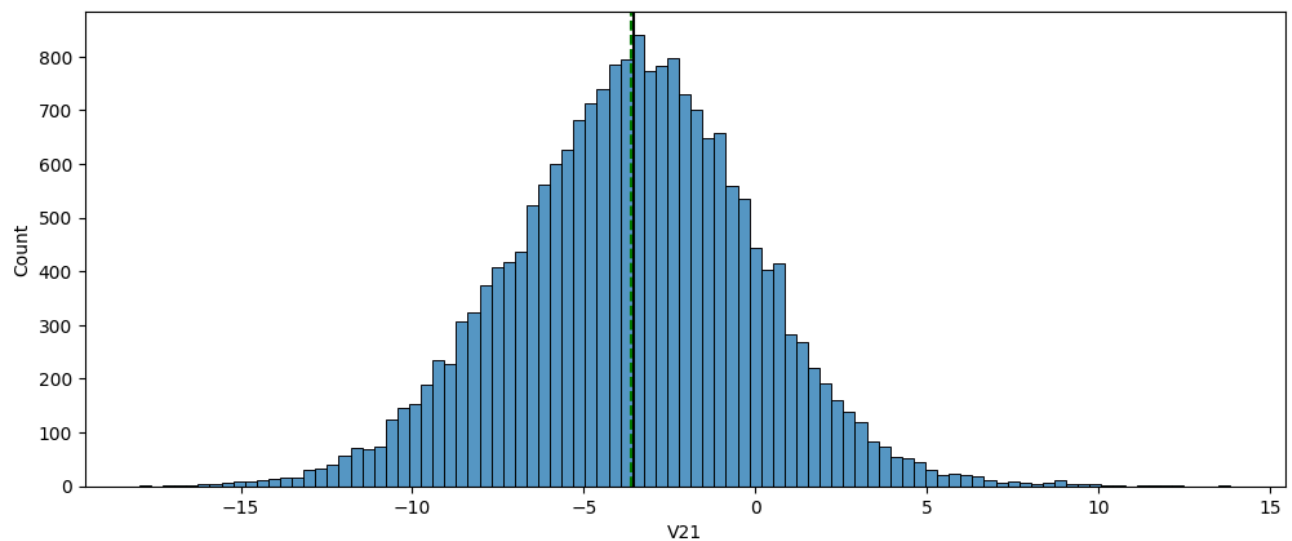


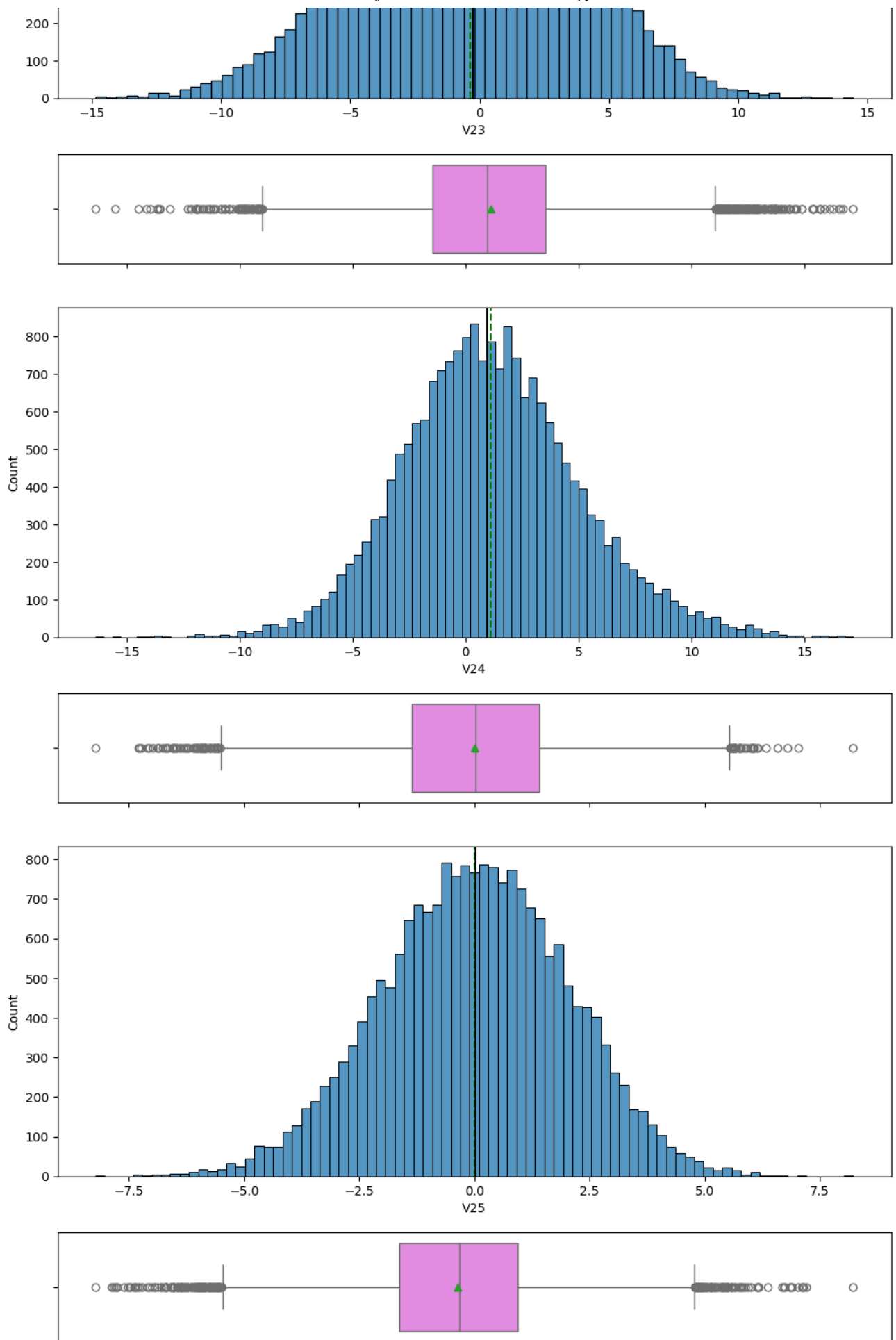


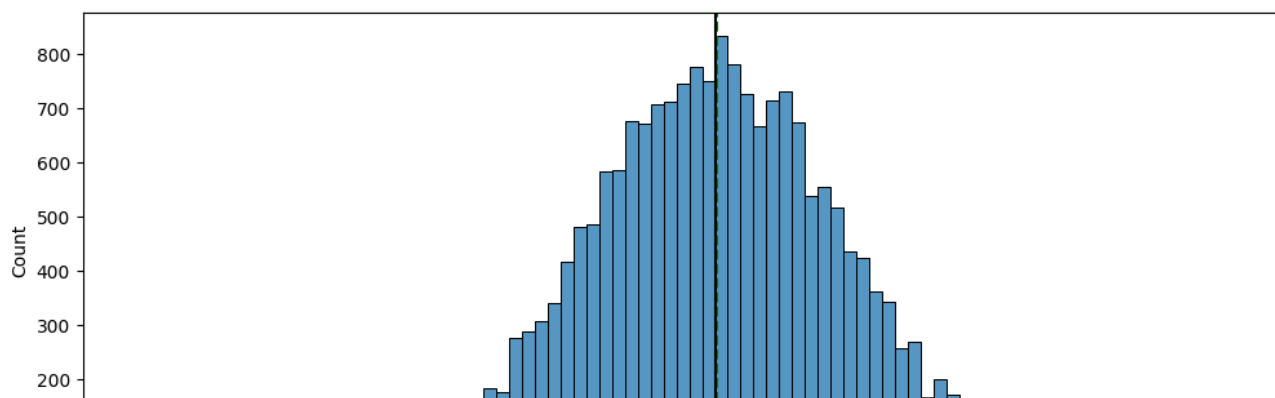
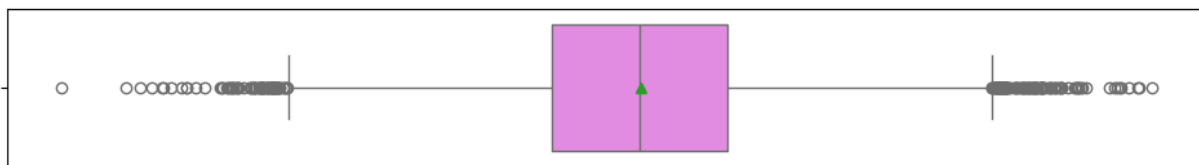
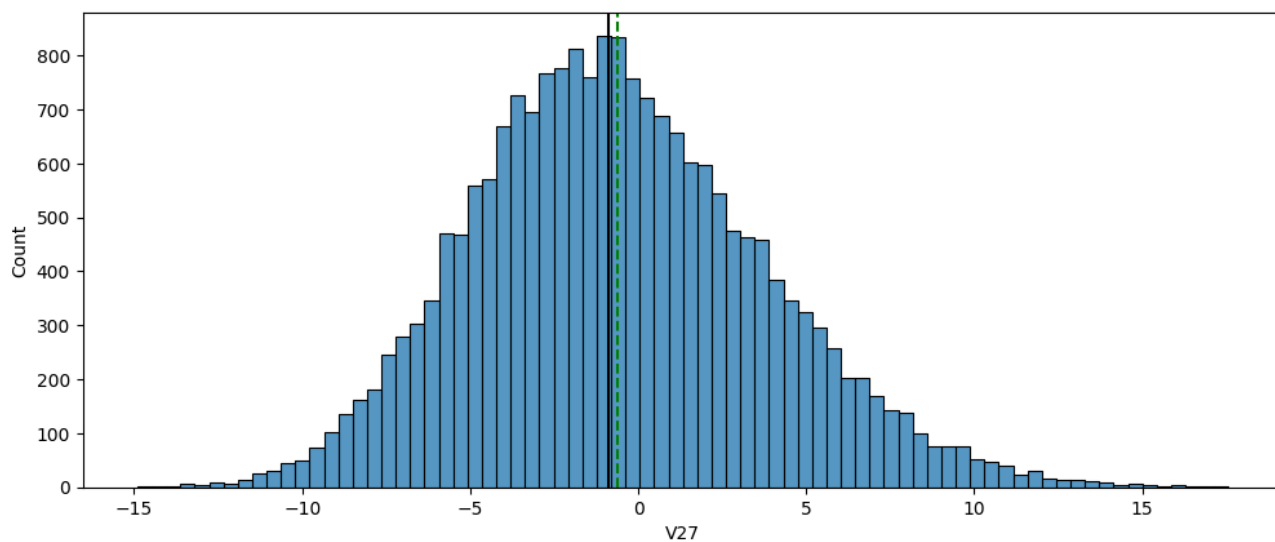
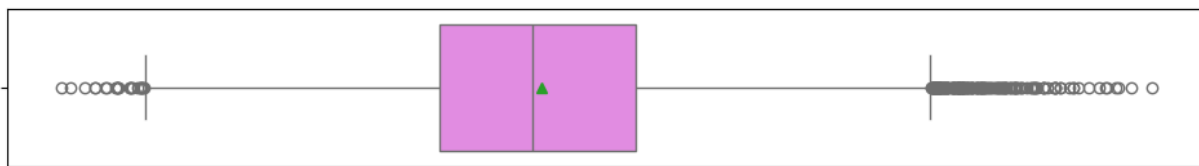
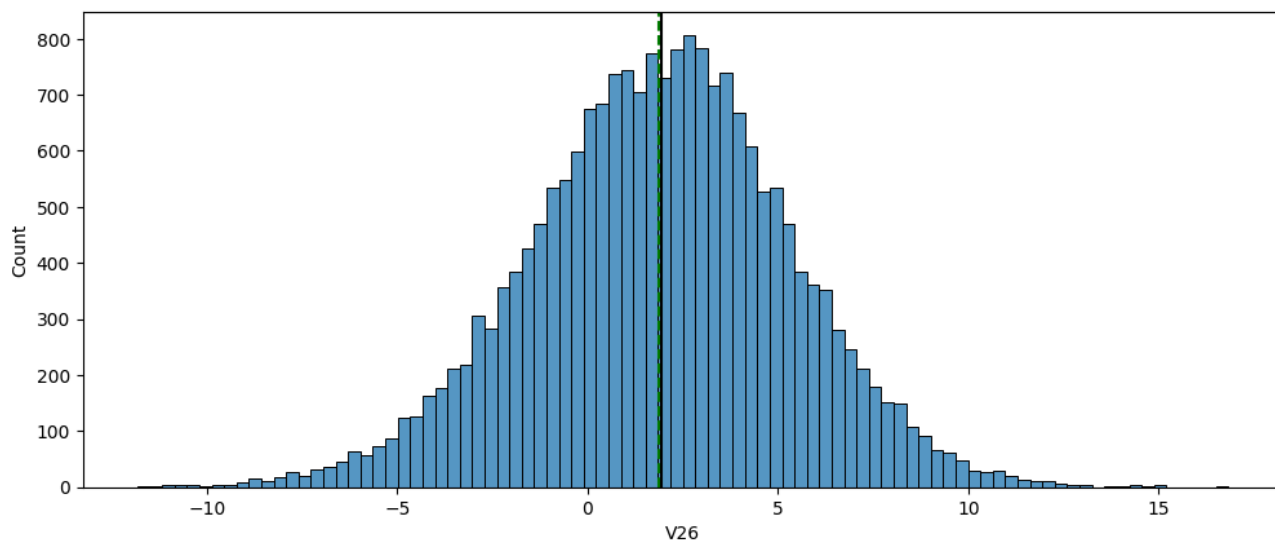


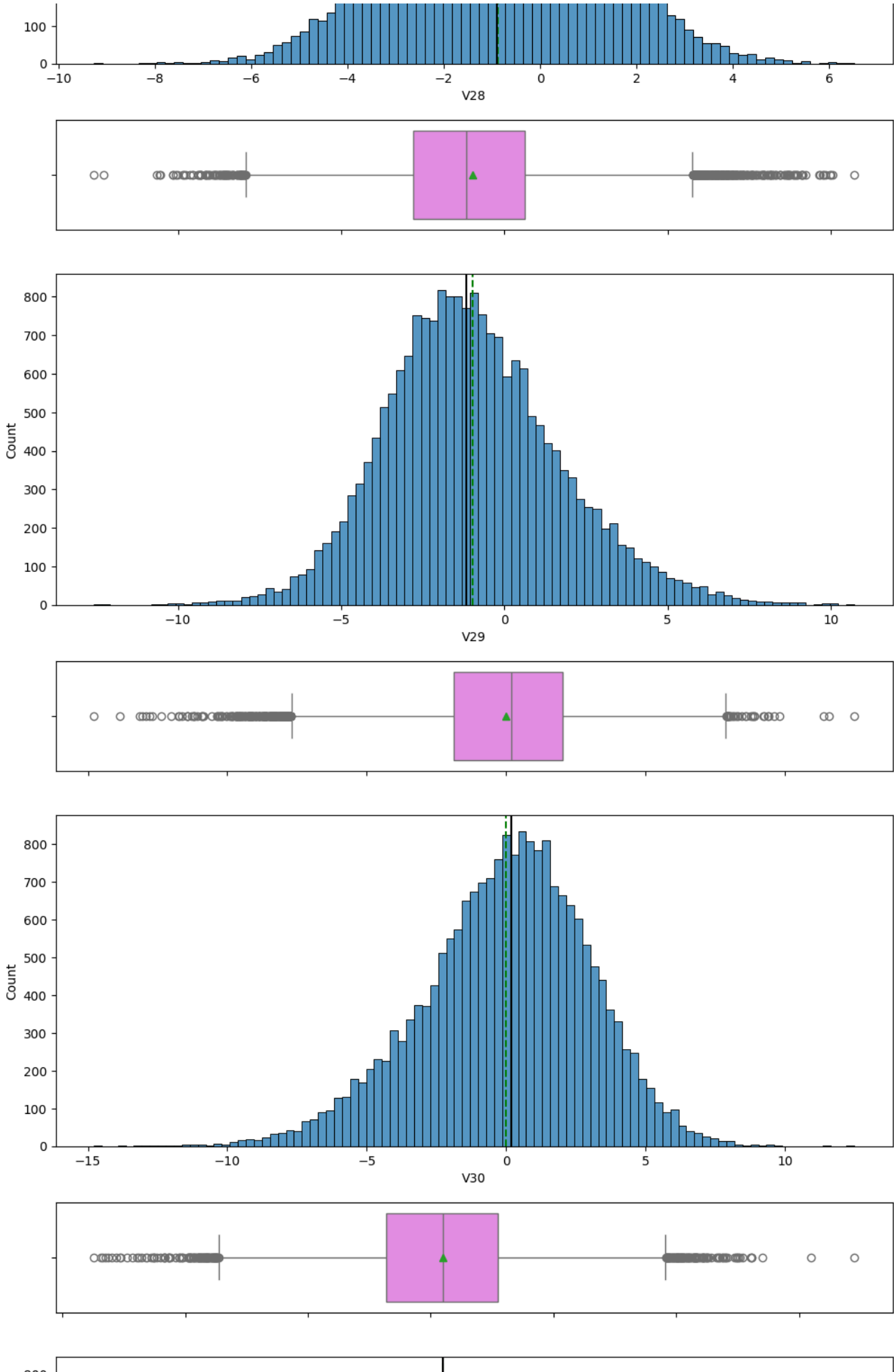


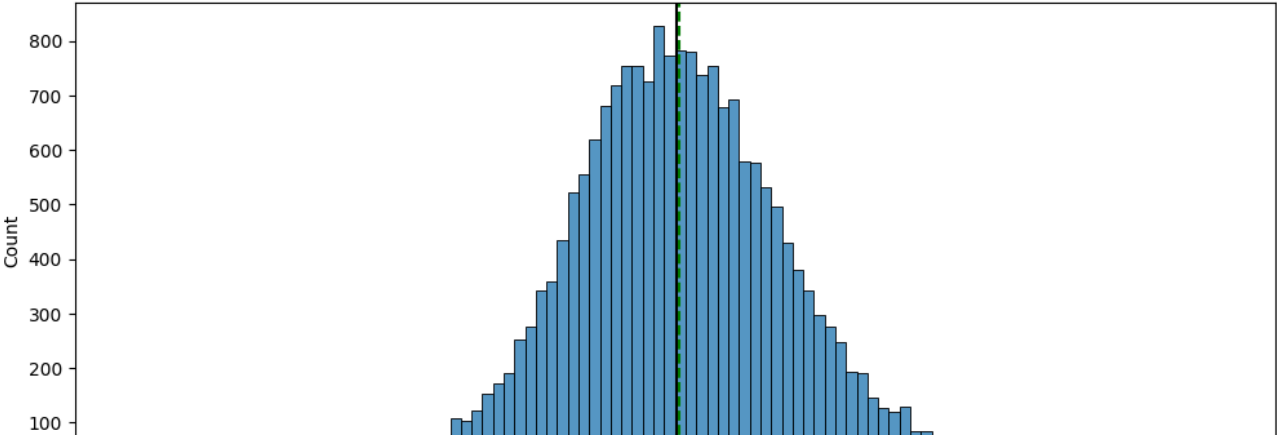
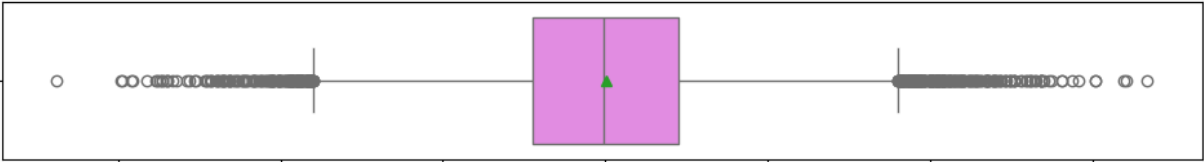
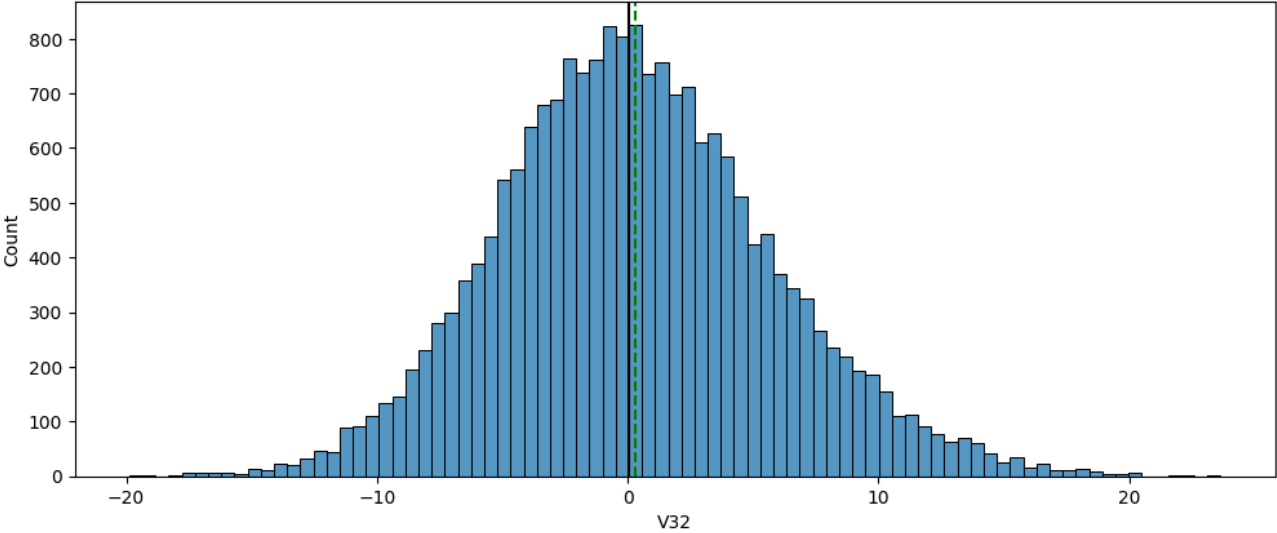
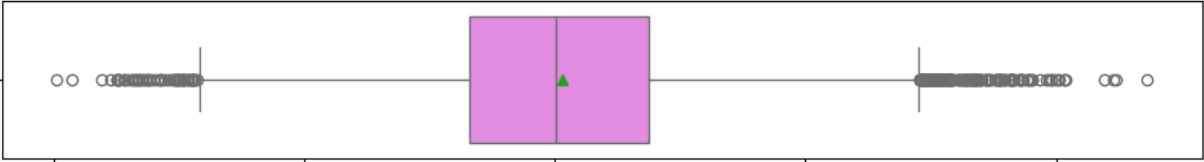
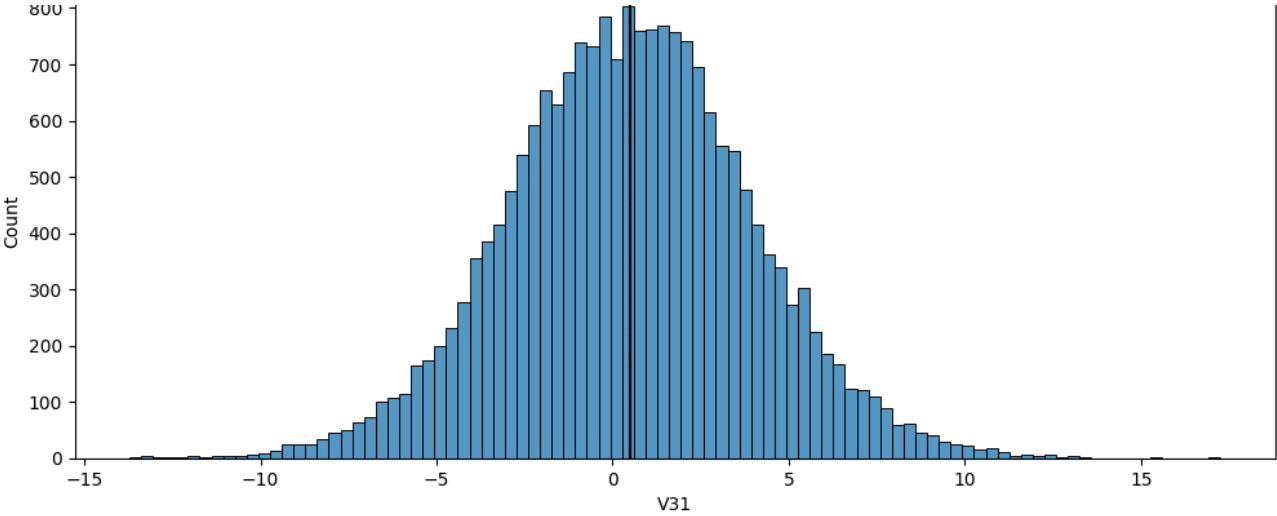


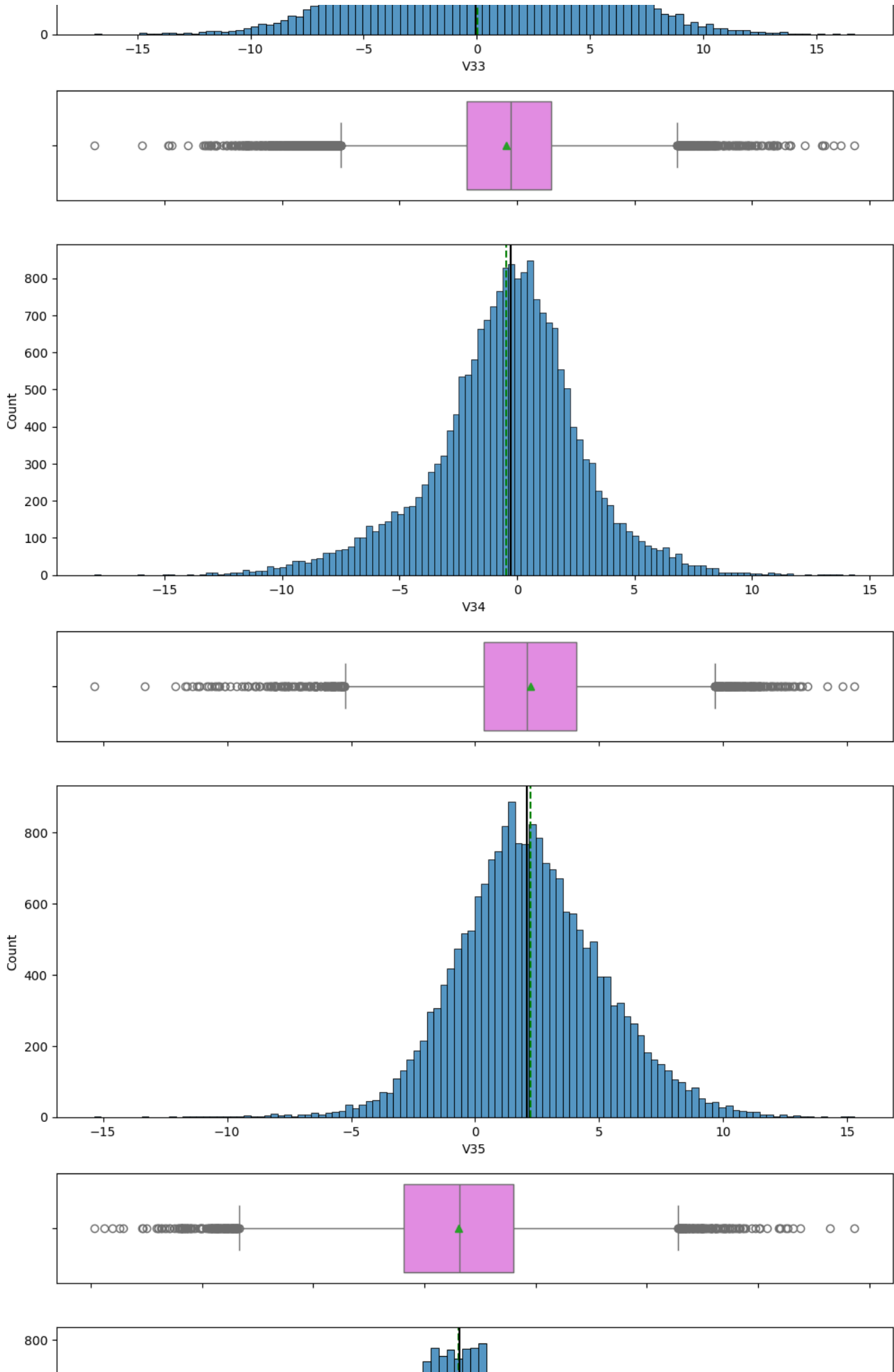


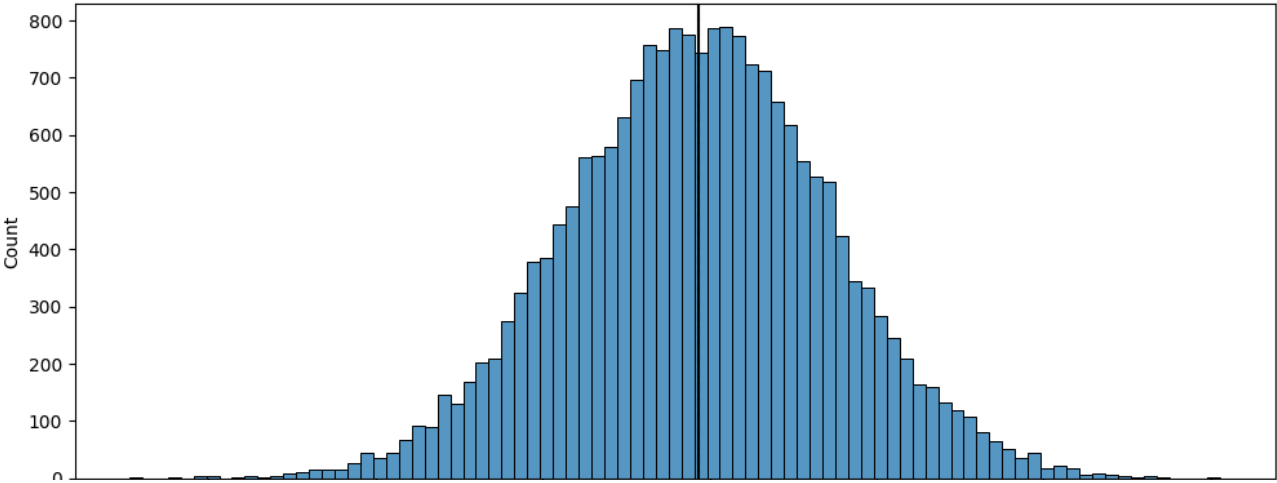
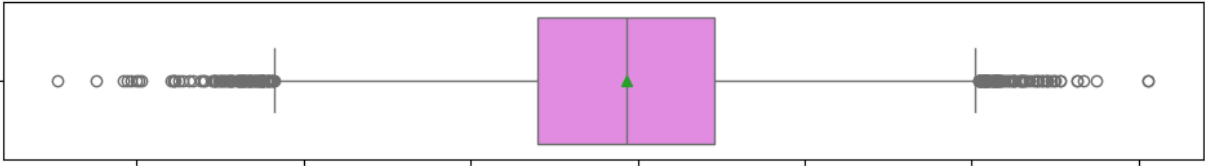
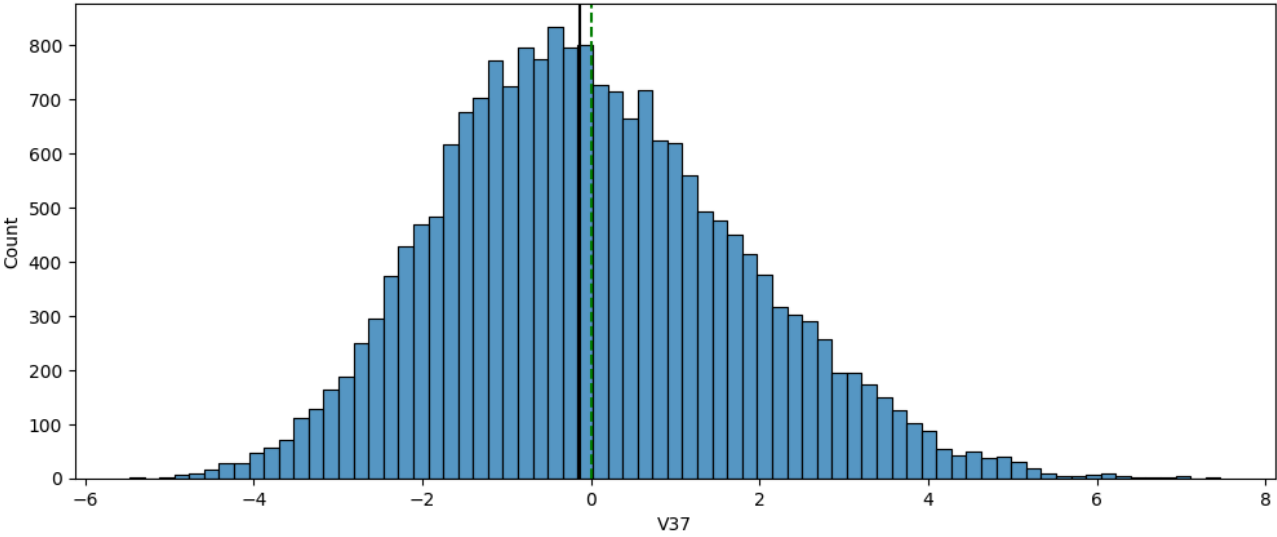
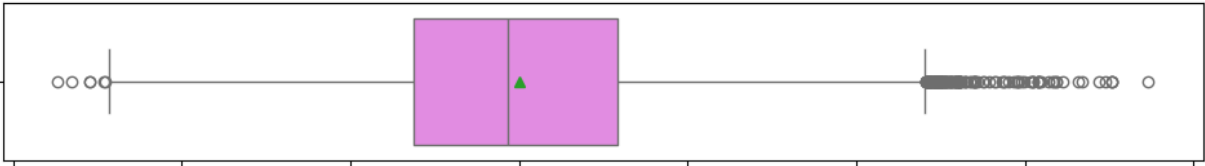
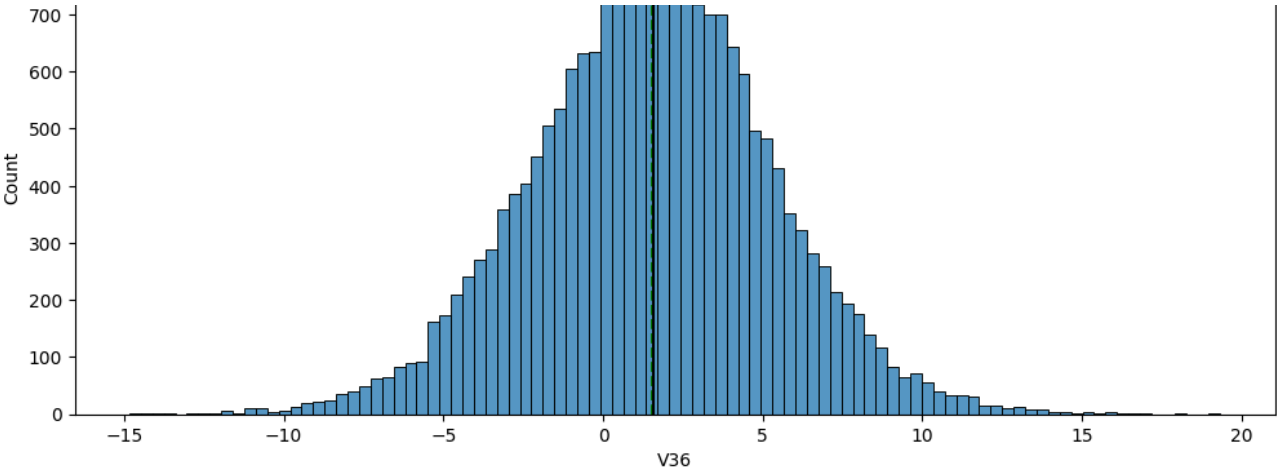


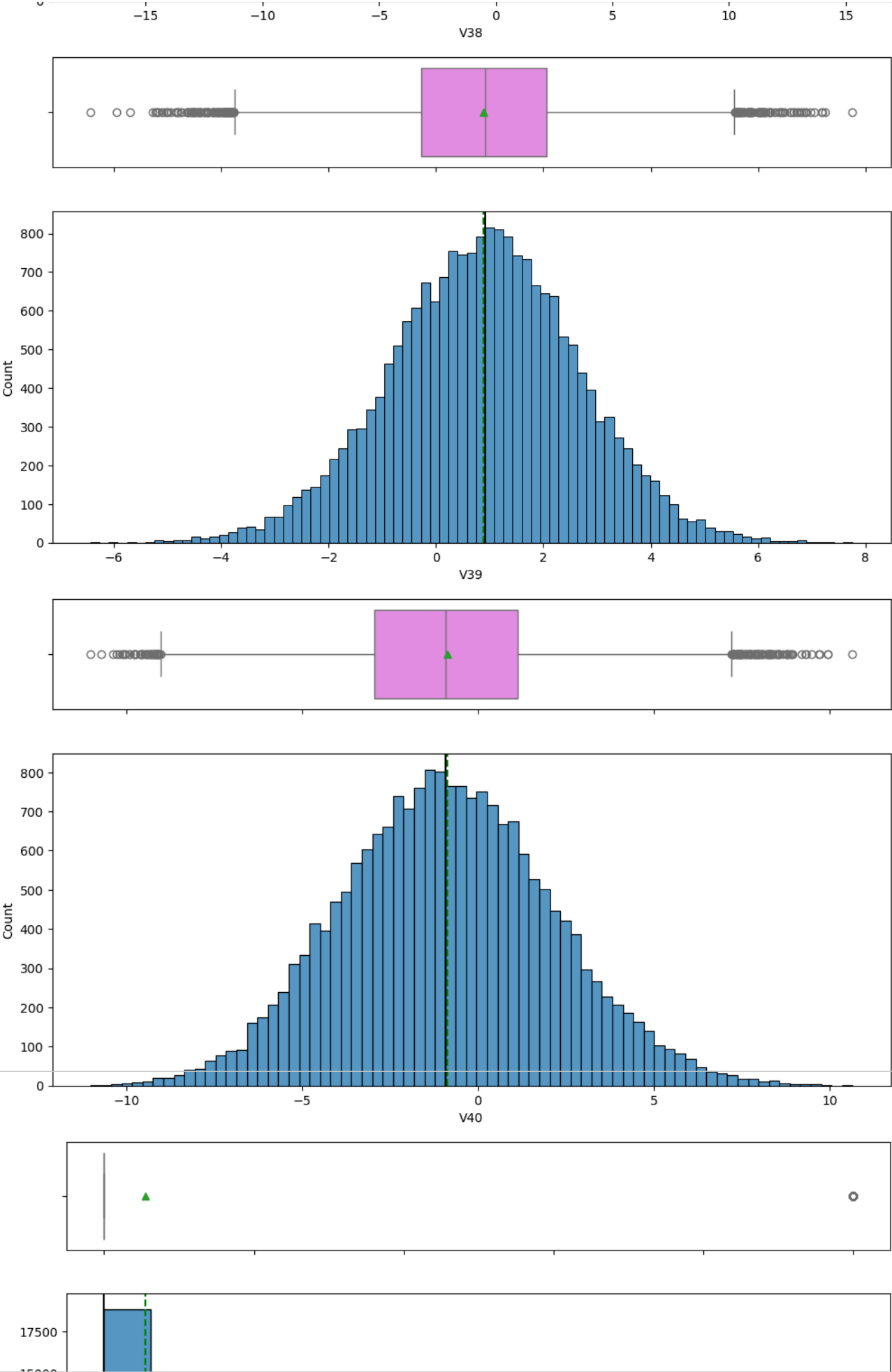






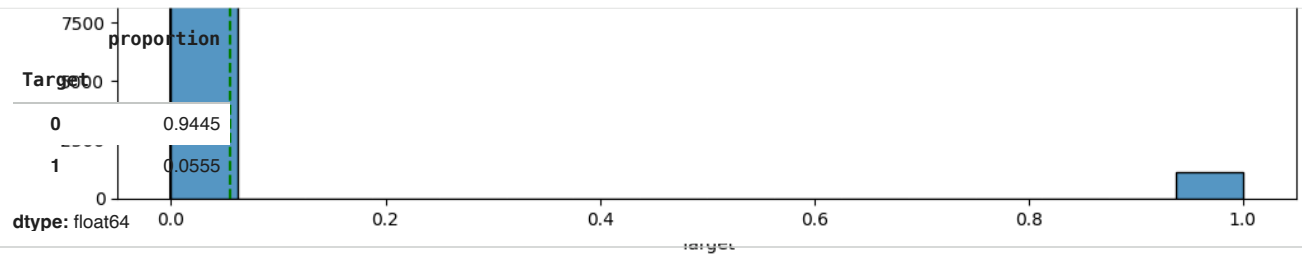






✓ Checking the distribution of Target variable

```
# For train data
df["Target"].value_counts(1)
```



```
# For test data
df_test["Target"].value_counts(1) # Complete the code to display the proportion of the target variable in the test d
```

```
proportion
Target
0      0.9436
1      0.0564

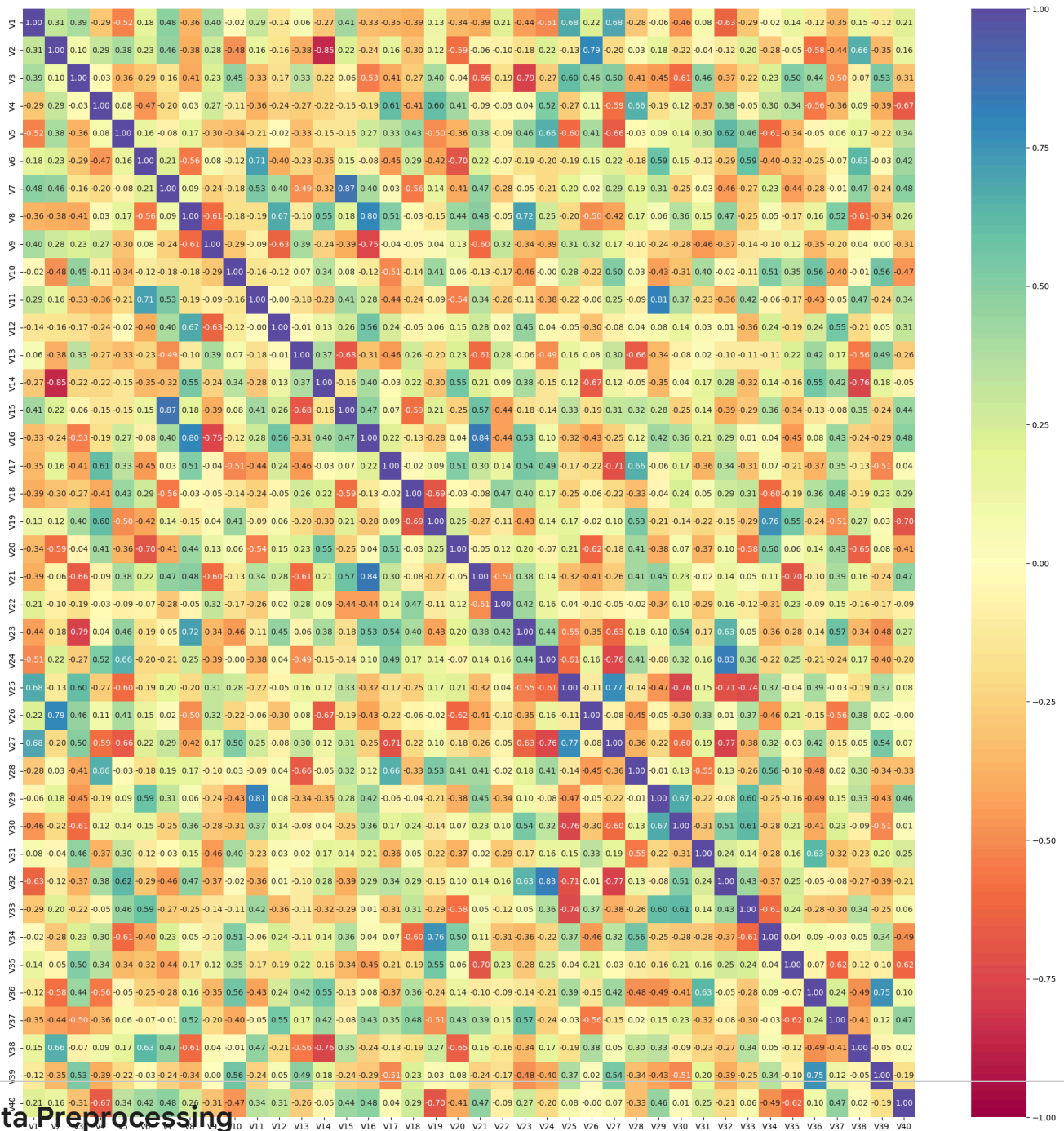
dtype: float64
```

✓ Bivariate Analysis

✓ Correlation Check

```
cols_list = df.select_dtypes(include=np.number).columns.tolist()
cols_list.remove("Target")

plt.figure(figsize=(25, 25))
sns.heatmap(
    df[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral"
)
plt.show()
```



▼ Data Preprocessing

▼ Data Preparation for Modeling

```
# Dividing train data into X and y
X = data.drop(columns = ["Target"], axis=1) # Complete the code to remove the column named 'Target'
y = data["Target"] # Complete the code to select the column named 'Target'
```

Since we already have a separate test set, we don't need to divide data into train, valiation and test

```
# Splitting data into training and validation set:

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=1, stratify=y # Complete the code to define the test size
)
```

```
# Checking the number of rows and columns in the X_train data
X_train.shape
```

```
(15000, 40)
```

```
# Checking the number of rows and columns in the X_val data
X_val.shape
```

```
(5000, 40)
```

```
# Dividing test data into X_test and y_test
X_test = data_test.drop(columns = ['Target'] , axis= 1) # Complete the code to remove the target column
y_test = data_test["Target"] # Complete the code to select the target column
```

```
# Checking the number of rows and columns in the X_test data
X_test.shape
```

```
(5000, 40)
```

✓ Missing Value Imputation

- There were few missing values in V1 and V2, we will impute them using the median.
- And to avoid data leakage we will impute missing values after splitting train data into train and validation sets.

```
imputer = SimpleImputer(strategy="median")
```

```
# Fit and transform the train data
X_train = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.columns)
```

```
# Transform the validation data
X_val = pd.DataFrame(imputer.transform(X_val), columns=X_train.columns) # Complete the code to impute missing val
```

```
# Transform the test data
X_test = pd.DataFrame(imputer.transform(X_test), columns=X_train.columns) # Complete the code to impute missing v
```

```
# Checking that no column has missing values in train or test sets
print(X_train.isna().sum())
print("-" * 30)
print(X_val.isna().sum())
print("-" * 30)
print(X_test.isna().sum())
```

```
V1      0
V2      0
V3      0
V4      0
V5      0
V6      0
V7      0
V8      0
V9      0
V10     0
V11     0
V12     0
V13     0
V14     0
V15     0
V16     0
V17     0
V18     0
V19     0
V20     0
V21     0
V22     0
V23     0
V24     0
V25     0
V26     0
V27     0
V28     0
V29     0
V30     0
V31     0
V32     0
```

```
V33    0
V34    0
V35    0
V36    0
V37    0
V38    0
V39    0
V40    0
dtype: int64
```

```
V1      0
V2      0
V3      0
V4      0
V5      0
V6      0
V7      0
V8      0
V9      0
V10     0
V11     0
V12     0
V13     0
V14     0
V15     0
V16     0
```

Model Building

Model Evaluation Criterion

- Write down the metric of choice with rationale here

We are now done with pre-processing and evaluation criterion, so let's start building the model.

Utility Functions

```
def plot(history, name):
    """
    Function to plot loss/accuracy

    history: an object which stores the metrics and losses.
    name: can be one of Loss or Accuracy
    """
    fig, ax = plt.subplots() #Creating a subplot with figure and axes.
    plt.plot(history.history[name]) #Plotting the train accuracy or train loss
    plt.plot(history.history['val_'+name]) #Plotting the validation accuracy or validation loss

    plt.title('Model ' + name.capitalize()) #Defining the title of the plot.
    plt.ylabel(name.capitalize()) #Capitalizing the first letter.
    plt.xlabel('Epoch') #Defining the label for the x-axis.
    fig.legend(['Train', 'Validation'], loc="outside right upper") #Defining the legend, loc controls the position o
```

```
# defining a function to compute different metrics to check performance of a classification model built using statsm
def model_performance_classification(
    model, predictors, target, threshold=0.5
):
    """
    Function to compute different metrics to check classification model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    threshold: threshold for classifying the observation as class 1
    """

    # checking which probabilities are greater than threshold
    pred = model.predict(predictors) > threshold
    # pred_temp = model.predict(predictors) > threshold
    # # rounding off the above values to get classes
    # pred = np.round(pred_temp)
```

```

acc = accuracy_score(target, pred) # to compute Accuracy
recall = recall_score(target, pred, average='macro') # to compute Recall
precision = precision_score(target, pred, average='macro') # to compute Precision
f1 = f1_score(target, pred, average='macro') # to compute F1-score

# creating a dataframe of metrics
df_perf = pd.DataFrame(
    {"Accuracy": acc, "Recall": recall, "Precision": precision, "F1 Score": f1,}, index = [0]
)

return df_perf

```

▼ Initial Model Building (Model 0)

- Let's start with a neural network consisting of
 - just one hidden layer of 7 neurons respectively
 - activation function of ReLU.
 - SGD as the optimizer

```

# defining the batch size and # epochs upfront as we'll be using the same values for all models
epochs = 50 # Complete the code to enter the number of epochs to be used in all models
batch_size = 32 # Complete the code to enter the batch size to be used in all models

```

```

# clears the current Keras session, resetting all layers and models previously created, freeing up memory and resour
tf.keras.backend.clear_session()

```

```

#Initializing the neural network
model_0 = Sequential()
model_0.add(Dense( 7 ,activation="relu",input_dim=X_train.shape[1])) # Complete the code to define the number of neu
model_0.add(Dense( 1 ,activation="sigmoid")) # Complete the code to define the number of neurons in the output layer

```

```
model_0.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 7)	287
dense_1 (Dense)	(None, 1)	8

Total params: 295 (1.15 KB)
Trainable params: 295 (1.15 KB)
Non-trainable params: 0 (0.00 B)

```

optimizer = tf.keras.optimizers.SGD() # defining SGD as the optimizer to be used
# model_0.compile(loss='binary_crossentropy', optimizer=optimizer, metrics = ['accuracy']) ## Uncomment this line in
# model_0.compile(loss='binary_crossentropy', optimizer=optimizer, metrics = ['Precision']) ## Uncomment this line i
model_0.compile(loss='binary_crossentropy', optimizer=optimizer, metrics = ['Recall'])
# model_0.compile(loss='binary_crossentropy', optimizer=optimizer, metrics = ['f1_score']) ## Uncomment this line in

```

```

start = time.time()
history = model_0.fit(X_train, y_train, validation_data=(X_val,y_val) , batch_size=batch_size, epochs=epochs)
end=time.time()

```

```

Epoch 1/50
469/469 ————— 2s 3ms/step - Recall: 0.3060 - loss: 0.2835 - val_Recall: 0.5072 - val_loss: 0.1342
Epoch 2/50
469/469 ————— 1s 2ms/step - Recall: 0.4865 - loss: 0.1207 - val_Recall: 0.5899 - val_loss: 0.1111
Epoch 3/50
469/469 ————— 1s 2ms/step - Recall: 0.6065 - loss: 0.1056 - val_Recall: 0.6259 - val_loss: 0.0993
Epoch 4/50
469/469 ————— 1s 2ms/step - Recall: 0.6648 - loss: 0.0896 - val_Recall: 0.6835 - val_loss: 0.0935
Epoch 5/50
469/469 ————— 1s 2ms/step - Recall: 0.6766 - loss: 0.0779 - val_Recall: 0.7122 - val_loss: 0.0895
Epoch 6/50
469/469 ————— 1s 2ms/step - Recall: 0.6976 - loss: 0.0838 - val_Recall: 0.7230 - val_loss: 0.0864
Epoch 7/50
469/469 ————— 1s 2ms/step - Recall: 0.7071 - loss: 0.0823 - val_Recall: 0.7446 - val_loss: 0.0837
Epoch 8/50
469/469 ————— 1s 2ms/step - Recall: 0.7278 - loss: 0.0750 - val_Recall: 0.7554 - val_loss: 0.0815

```

```

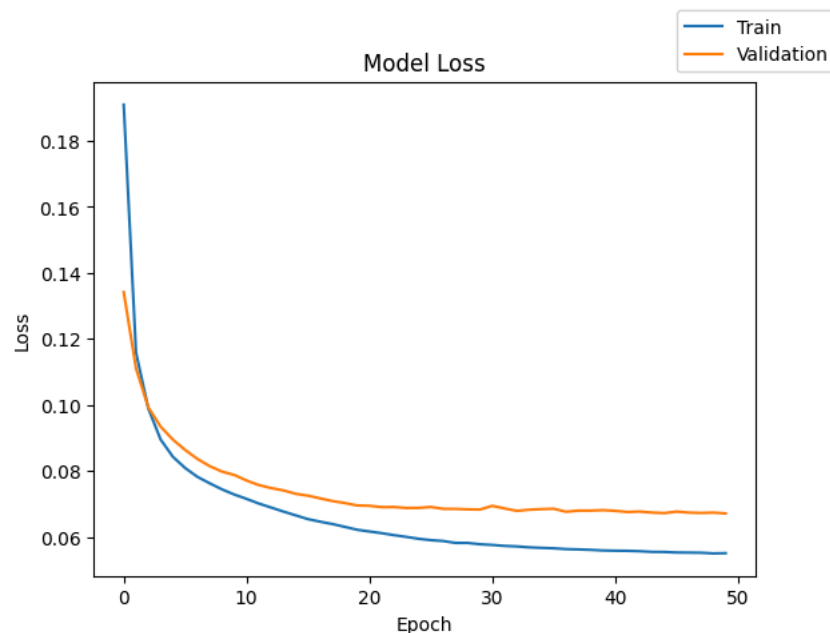
Epoch 9/50
469/469 ————— 1s 2ms/step - Recall: 0.7005 - loss: 0.0741 - val_Recall: 0.7590 - val_loss: 0.0798
Epoch 10/50
469/469 ————— 2s 3ms/step - Recall: 0.7406 - loss: 0.0737 - val_Recall: 0.7626 - val_loss: 0.0788
Epoch 11/50
469/469 ————— 1s 3ms/step - Recall: 0.7315 - loss: 0.0709 - val_Recall: 0.7842 - val_loss: 0.0771
Epoch 12/50
469/469 ————— 1s 2ms/step - Recall: 0.7631 - loss: 0.0721 - val_Recall: 0.7770 - val_loss: 0.0757
Epoch 13/50
469/469 ————— 1s 2ms/step - Recall: 0.7739 - loss: 0.0655 - val_Recall: 0.7806 - val_loss: 0.0748
Epoch 14/50
469/469 ————— 1s 2ms/step - Recall: 0.7331 - loss: 0.0741 - val_Recall: 0.7806 - val_loss: 0.0741
Epoch 15/50
469/469 ————— 1s 2ms/step - Recall: 0.7506 - loss: 0.0619 - val_Recall: 0.7950 - val_loss: 0.0731
Epoch 16/50
469/469 ————— 1s 2ms/step - Recall: 0.7732 - loss: 0.0642 - val_Recall: 0.7986 - val_loss: 0.0725
Epoch 17/50
469/469 ————— 1s 2ms/step - Recall: 0.7741 - loss: 0.0646 - val_Recall: 0.8022 - val_loss: 0.0717
Epoch 18/50
469/469 ————— 1s 2ms/step - Recall: 0.7746 - loss: 0.0679 - val_Recall: 0.7986 - val_loss: 0.0709
Epoch 19/50
469/469 ————— 1s 2ms/step - Recall: 0.7778 - loss: 0.0639 - val_Recall: 0.7986 - val_loss: 0.0703
Epoch 20/50
469/469 ————— 1s 2ms/step - Recall: 0.8001 - loss: 0.0643 - val_Recall: 0.7986 - val_loss: 0.0696
Epoch 21/50
469/469 ————— 1s 2ms/step - Recall: 0.7968 - loss: 0.0622 - val_Recall: 0.8094 - val_loss: 0.0695
Epoch 22/50
469/469 ————— 2s 4ms/step - Recall: 0.7688 - loss: 0.0654 - val_Recall: 0.7986 - val_loss: 0.0691
Epoch 23/50
469/469 ————— 1s 2ms/step - Recall: 0.7941 - loss: 0.0597 - val_Recall: 0.8094 - val_loss: 0.0691
Epoch 24/50
469/469 ————— 1s 2ms/step - Recall: 0.7784 - loss: 0.0648 - val_Recall: 0.8058 - val_loss: 0.0688
Epoch 25/50
469/469 ————— 1s 2ms/step - Recall: 0.7925 - loss: 0.0613 - val_Recall: 0.8094 - val_loss: 0.0688
Epoch 26/50
469/469 ————— 1s 2ms/step - Recall: 0.7550 - loss: 0.0666 - val_Recall: 0.8165 - val_loss: 0.0691
Epoch 27/50
469/469 ————— 1s 2ms/step - Recall: 0.8052 - loss: 0.0586 - val_Recall: 0.8094 - val_loss: 0.0685
Epoch 28/50
469/469 ————— 1s 2ms/step - Recall: 0.8010 - loss: 0.0587 - val_Recall: 0.8022 - val_loss: 0.0685
Epoch 29/50
469/469 ————— 1s 2ms/step - Recall: 0.7866 - loss: 0.0588 - val_Recall: 0.8004 - val_loss: 0.0684

```

```
print("Time taken in seconds ",end-start)
```

```
Time taken in seconds 58.282419204711914
```

```
plot(history,'loss')
```



```

model_0_train_perf = model_performance_classification(model_0, X_train, y_train)
model_0_train_perf

```